

Minimization of expensive functions in the presence of forbidden regions *

F. A. Fortunato [†] J. M. Martínez [†]

September 18, 2026

Abstract

In estimating parameters for models that reflect the spatiotemporal evolution of a physical system, minimizing functions that involve prediction errors incurs a high computational cost because each evaluation requires solving a typically expensive “direct problem”. Furthermore, these functions may be undefined for parameter sets whose description via equations or inequalities is impossible. This paper addresses both challenges. To handle minimization with forbidden points, we define a general algorithm. On the other hand, to cope with the high cost of function evaluations, we propose a quartic algorithm, built from quadratic models of each residual, minimized as a surrogate at each iteration, whose cost is affordable compared to evaluating the objective function and gradient. The proposed algorithm is first tested on the classical MGH least-squares problems, used to select its internal configuration, and then applied to the problem of estimating the hydraulic parameters in the equations governing flow behavior in natural channels. In both cases, the proposed algorithm is shown to be competitive with classical methods.

Key words: nonlinear least squares, hidden constraints, quasi-Newton methods, Manning coefficient, river flow modeling

1 Introduction

We are interested in optimization problems that arise from estimating the parameters of time-evolving systems, typically obtained after discretizing a system of partial differential equations posed on a space-time domain. Problems of this type are tackled in several ways. The most widely used approaches rely on grid search, on metaheuristics, or on statistical methods in the style of Monte Carlo simulation [1, 23, 24], all of which tend to require a large number of function evaluations. Other approaches exist to overcome this drawback, but here we focus on those that exploit the structure of the underlying optimization problem itself to accelerate the parameter-estimation process, as is done, for instance, in [6, 19, 32].

In problems of this kind, evaluating the objective function is generally expensive, because each evaluation requires solving the underlying system. The objective function is typically a sum of squares of the discrepancies with respect to the available observations, and its gradient can generally be computed using automatic differentiation tools.

On the other hand, for some choices of the parameters to be estimated, the system may not be well defined (it may blow up), and it is not possible to foresee a priori in which regions this numerical event may occur. Some studies treat the problem as a whole, as one with hidden constraints, and such methods are generally derivative-free [2, 3, 8, 13, 20, 25, 38]. Placing the unknown constraint directly inside the line search itself can also be done either way with a derivative-free structure [16, 22], interpolation and/or with a derivative-based formulation, as in the line searches implemented in LineSearches.jl [30]. When

*This work was supported by PRONEX-Optimization (PRONEX - CNPq / FAPERJ E-26 / 171.510/2006 - APQ1), FAPESP (Grants 2006/53768-0 and 2004/15635-2) and CNPq.

[†]Department of Applied Mathematics, IMECC-UNICAMP, University of Campinas, CP 6065, 13081-970 Campinas SP, Brazil. e-mail: fabio@ime.unicamp.br, martinez@ime.unicamp.br

the trial point of an iterative minimization method turns out to be a forbidden point, in the sense that the objective function is undefined there, the usual practice is to step the trial point back toward the current point, trusting that the search process will encounter these anomalies only infrequently.

Clearly, interpolation strategies for this backtracking are prohibitive, since there is no objective function value at the trial point that can be used. Furthermore, if the user instead assigns it an arbitrarily large but finite value, cubic or quadratic interpolation lands on points whose distance to the current point is negligible, thereby yielding little or no useful information. In `Optim.jl` [30], for instance, this interpolation-based backtracking only behaves correctly if the objective function is defined to return $+\infty$, rather than some large finite value, at a forbidden point. This workaround is not needed in the second approach we propose (Section 4).

This backtracking strategy is enough to keep the iterates away from forbidden points, but it makes no further use of the function and gradient values already computed at the rejected trial points, values that, in the setting described above, were themselves costly to obtain. When function and gradient evaluations are cheap, discarding this information is harmless. When they are expensive, as is the case throughout this paper, it is wasteful, since the same evaluations could instead be used to build a more informative model of the objective near the current point, at a cost that is negligible by comparison with a single evaluation of the objective and its gradient.

As noted above, the derivative-free treatment of hidden constraints is generally based on an extreme-barrier approach. Here we extend this idea to a gradient-based line search, with a convergence guarantee that needs no Lipschitz or Hölder assumption on ∇f . On the complexity side, the $O(\varepsilon^{-2})$ bound we establish for Algorithm 4.1 matches the classical worst-case rate known for the (unregularized) Levenberg-Marquardt method for nonlinear least squares [28, 41]. Sharper rates, such as the $O(\varepsilon^{-3/2})$ bound of adaptive cubic regularization [10, 11], are known for general unconstrained minimization, but, to our knowledge, not yet under the weaker per-residual assumptions and forbidden-region setting considered here.

The contributions of this paper are as follows. First, we propose a general line-search framework for unconstrained minimization in the presence of forbidden points, and prove its convergence under no Lipschitz or Hölder assumption on the gradient (Section 3). Together with the discussion above, this addresses a gap in the theoretical study of methods for hidden constraints, which to date have been treated mostly by heuristic derivative-free rules, such as the extreme-barrier approach, with comparatively little accompanying convergence theory. Second, for the case in which the objective and its gradient are expensive to evaluate, we propose Algorithm 4.1, a regularized quartic-model method for nonlinear least squares that reuses per-residual quasi-Newton Hessians across iterations, and we establish its well-posedness and an $O(\varepsilon^{-2})$ worst-case complexity bound under assumptions on the residuals alone (Section 4). Unlike a line search, Algorithm 4.1 never fixes a direction and rescales it along a ray. A rejected trial step only increases the regularization parameter σ and re-solves the resulting subproblem, so no line search of any kind is required. It is also, by construction, independent of the value assigned to a forbidden point, since feasibility is checked directly, before that value could ever enter any comparison. The same guarantees therefore hold whether a forbidden point is represented by $+\infty$ or by an arbitrarily large finite penalty, unlike the interpolation-based line searches discussed above.

This way, the paper is organized as follows. Sections 2 and 3 formalize the first part of this framework, for a general smooth unconstrained objective. Section 2 defines what a forbidden region is, and Section 3 proposes Algorithm 3.1, a line-search method that treats a forbidden trial point exactly as a rejected one, and proves its convergence, requiring no Lipschitz or Hölder condition on the gradient. Section 4 then replaces the backtracking step of Algorithm 3.1 with the minimization, at each iteration, of a regularized quartic surrogate built from quasi-Newton approximations to the Hessian of each residual, giving Algorithm 4.1. Under a Lipschitz-type condition on the residual gradients, we prove that Algorithm 4.1 is well defined and establish a worst-case complexity bound for reaching an approximate stationary point.

Section 5 deals with a degeneracy that arises at Algorithm 4.1’s very first iteration, where no quasi-Newton curvature has yet been observed and the quartic subproblem collapses to a damped Gauss–Newton step. We show that this step remains a genuine descent direction for every value of the regularization parameter and becomes asymptotically equivalent to a steepest-descent step as that parameter grows, so

no separate safeguard is required.

In Section 6 we present numerical results on the classical Moré-Garbow-Hillstom sum-of-squares test problems [31], which we use both to calibrate the internal choices of Algorithm 4.1 (Hessian-update formula and quartic-subproblem solver) and to compare it against BFGS [33, 36].

In Section 7, we apply Algorithm 4.1, together with BFGS and the derivative-free BOBYQA [37, 38], to a parameter-estimation problem in natural channels, calibrating the Manning roughness coefficient of the Saint-Venant equations against elevation observations.

Finally, in Section 8, we present our conclusions and outline directions for future research.

Notation

- $\|\cdot\|$ denotes the Euclidean norm.
- $A - B$ denotes $\{x \in A \mid x \notin B\}$.
- $\mathcal{B}(x, \delta) = \{z \in \mathbb{R}^n \mid \|x - z\| \leq \delta\}$.

2 Optimization in the presence of forbidden sets

Many optimization problems involve forbidden regions. This means parts of the domain where the objective function is not defined, is unreliable, or has no physical meaning. Sometimes these regions are excluded by introducing constraints to the problem, but on other occasions, resorting to constraints does not work. On the one hand, forbidden regions may not be well-characterized by constraints defined by algebraic equalities or inequalities. On the other hand, even when safe constraints can be identified, algorithms tend to hover near their infeasibility, eagerly searching for low values of the objective function, when the right move would be to abandon the forbidden region forcefully and, at the same time, sensibly. Other times, especially in the field of derivative-free optimization, algorithms include heuristic rules to avoid forbidden regions.

{forbid}

The algorithmic framework addressed in this paper is simple: we recognize that well-established algorithms exist for solving well-defined optimization problems, and we try to leverage their features as much as possible, instead of discarding them to look for radically different alternatives. The efficiency of this procedure lies in the modeler's ability to detect forbidden points. So, the basic method consists of adopting a well-established algorithm—accepting that, occasionally, it may lead us to forbidden points—and treating those forbidden points as if they were points where the objective function is very large, but without using their value (which does not even need to be calculated or exist) for future computations. It is important to highlight that a strong motivation for employing forbidden region techniques is that objective functions can be very expensive to evaluate and, therefore, it is important to have criteria to avoid evaluating them when their evaluation promises to be useless.

The typical situation in which one detects that the point x , where one wishes to evaluate the objective function, is a forbidden point occurs when solving a system of differential equations and, at a certain time instant, the approximate solution blows up. In this case, one must interrupt the solution of the system and mark the point in question as forbidden. It should be noted that, in general, the time required to detect that a point is forbidden is shorter than the time to perform a full function evaluation, precisely because this evaluation has usually been interrupted in such cases.

Standard unconstrained minimization algorithms proceed, at each iteration, by calculating trial points that are accepted or rejected according to the value of the objective function. Even when rejected, these trial points contribute to the selection of subsequent trial points because they are frequently used in interpolation procedures that replace (simpler) backtracking steps.

3 Line-search algorithm

Consider the problem

$$\text{Minimize } f(x) \text{ subject to } x \in \mathbb{R}^n \text{ and } x \notin P, \quad (1) \quad \{\text{problem}\}$$

where $P \subset \mathbb{R}^n$ is closed.

We assume that $\nabla f(x)$ exists and is continuous for all $x \in \mathbb{R}^n - P$. We denote $g(x) = \nabla f(x)$.

P will be assumed to be the set of “forbidden points”. Given $x \in \mathbb{R}^n$, we assume that we can decide, in finite time, if x belongs to P or not.

Algorithm 3.1 is an adaptation of a classical approach for smooth unconstrained minimization to the case in which forbidden points exist. See, for example, Theorem 8.2 of [7].

Algorithm 3.1.

Let $\theta \in (0, 1)$, $\alpha \in (0, 1/2)$, and $\beta > 0$ be algorithmic parameters. Let $x^0 \in \mathbb{R}^n - P$ be the initial approximation.

Step 0. Initialize $k \leftarrow 0$.

Step 1. If $\|g(x^k)\| = 0$, finish the execution of the algorithm.

Step 2. Compute $d^k \in \mathbb{R}^n$ such that

$$g(x^k)^T d^k \leq -\theta \|d^k\| \|g(x^k)\| \text{ and } \|d^k\| \geq \beta \|g(x^k)\|. \quad (2) \quad \{\text{conds}\}$$

Step 3. Define

$$t \leftarrow 1. \quad (3) \quad \{\text{tuno}\}$$

Step 4. If

$$x^k + td^k \notin P \quad (4) \quad \{\text{notin}\}$$

and

$$f(x^k + td^k) \leq f(x^k) + \alpha t g(x^k)^T d^k, \quad (5) \quad \{\text{armijo}\}$$

define $t_k = t$, $x^{k+1} = x^k + t_k d^k$. Set $k \leftarrow k + 1$, and go to Step 1.

Step 5. Choose $t_{new} \in [0.1t, 0.9t]$, set $t \leftarrow t_{new}$, and go to Step 4.

Lemma 3.1. *Every iteration of Algorithm 3.1 is well defined.*

Proof. By definition $x^0 \notin P$. Then, $g(x^k)$ exists and the algorithm stops at x^k if and only if $\|g(x^k)\| = 0$.

Assume that $x^k \notin P$ has been computed and $\|g(x^k)\| \neq 0$. Then, by (2),

$$g(x^k)^T d^k < 0. \quad (6) \quad \{\text{con}\}$$

We will prove that, after a finite number of inner steps, we can find $t > 0$ such that $x^k + td^k \notin P$ and (5) takes place.

Then, there exists $\bar{t} > 0$ such that $\|(x^k + td^k) - x^k\| \leq \delta$ for all $t \in [0, \bar{t}]$. But, after a final number of reductions of t at Step 4 we have that $t \leq \bar{t}$, so the condition (4) takes place for all $t \leq \bar{t}$. Then, by (6), the definition of directional derivative and the continuity of the gradient, we have that

$$\lim_{t \rightarrow 0} \frac{f(x^k + td^k) - f(x^k)}{t} = g(x^k)^T d^k < 0$$

So, since $\alpha < 1$, for t small enough we have that

$$\frac{f(x^k + td^k) - f(x^k)}{tg(x^k)^T d^k} \geq \alpha.$$

Therefore, (5) holds for t small enough. So, after a finite number of reductions of t , the acceptance condition (5) takes place. This completes the proof. $\square$

The convergence theorem related with Algorithm 3.1 follows below. Note that this theorem does not use Lipschitz or Hölder conditions on ∇f .

Theorem 3.1. *If x^* is a limit point of the sequence generated by Algorithm 3.1, we have that $x^* \in P$ or $\|g(x^*)\| = 0$.*

Proof. Let $K \subset \mathbb{N}$ be an infinite sequence of indices such that

$$\lim_{k \in K} x^k = x^*. \quad (7) \quad \{\text{limK}\}$$

If $x^* \in P$ there is nothing to prove. So, from now on we assume that $x^* \notin P$. Since $\mathbb{R}^n - P$ is open, there exists $\delta > 0$ such that

$$x^k \in \mathcal{B}(x^*, \delta) \subset \mathbb{R}^n - P \text{ for all } k \in K. \quad (8) \quad \{\text{dentro}\}$$

Suppose that

$$\|g(x^*)\| = c > 0. \quad (9) \quad \{\text{absu}\}$$

We will prove that (9) is impossible.

Let us consider two cases:

Case 1: $\lim_{k \in K} \|t_k d^k\| = 0$.

Case 2: There exists a subsequence $K_1 \subset K$ and $c_1 > 0$ such that $\|t_k d^k\| > c_1$ for all $k \in K_1$.

Clearly, Case 2 is the negation of Case 1.

Assume that Case 1 takes place.

By (8) and (7), as $\|t_k d^k\|$ tends to 0 for $k \in K$, we have that, for $k \in K$ large enough,

$$x^k + t d^k \in \mathcal{B}(x^*, \delta) \subset \mathbb{R}^n - P \text{ whenever } t \in [0, 10t_k]. \quad (10) \quad \{\text{diez}\}$$

Therefore, for $k \in K$ large enough, the choice of t_k has been preceded by a choice of $t'_k \leq 10t_k$ such that (5) failed to hold when $t = t'_k$. Moreover, since $\lim_{k \in K} \|t_k d^k\| = 0$ we have that

$$\lim_{k \in K} \|t'_k d^k\| = 0. \quad (11) \quad \{\text{liteli}\}$$

So, for $k \in K$ large enough,

$$f(x^k + t'_k d^k) > f(x^k) + \alpha t'_k g(x^k)^T d^k \quad (12) \quad \{\text{noarm}\}$$

and

By (12) and the Mean Value Theorem, for all $k \in K$ large enough, there exists $\xi_k \in [0, 1]$ such that

$$g(x^k + \xi_k t'_k d^k)^T d^k > \alpha g(x^k)^T d^k. \quad (13) \quad \{\text{tvm}\}$$

By (11), dividing both members of (13) by $\|d^k\|$ and taking limits for $k \in K$ at a convergent subsequence of $d^k/\|d^k\|$ we get:

$$g(x^*)^T d \geq \alpha g(x^*)^T d \quad (14) \quad \{\text{thico}\}$$

where $\|d\| = 1$ and d is the limit of $d^k/\|d^k\|$ for a convergent subsequence. Since $0 < \alpha < 1$, this implies that

$$g(x^*)^T d \geq 0. \quad (15) \quad \{\text{thic}\}$$

But, by (2), we have that

$$g(x^k)^T d^k \leq -\theta \|g(x^k)\| \|d^k\|,$$

So,

$$g(x^*)^T d \leq -\theta \|g(x^*)\| < 0.$$

This contradicts (15). This contradiction comes from assuming that Case 1 takes place.

Therefore, we only need to analyze Case 2. By the definition of the algorithm we have that $x^k + t_k d^k \notin P$. Moreover, by (5),

$$f(x^k + t_k d^k) \leq f(x^k) + \alpha t_k g(x^k)^T d^k.$$

So, by (2),

$$f(x^k + t_k d^k) \leq f(x^k) - \alpha \theta t_k \|g(x^k)\| \|d^k\|.$$

So, under the assumption that $\|g(x^*)\| > 0$, for $k \in K$ large enough we have that

$$f(x^k + t_k d^k) \leq f(x^k) - \alpha \theta c c_1. \quad (16) \quad \{\text{abajó}\}$$

Since $f(x^{k+1}) \leq f(x^k)$ for all $k \in \mathbb{N}$, (16) implies that $\lim f(x^k) = -\infty$, which contradicts the continuity of f at x^* . So, Case 2 is impossible too. Since both possibilities are impossible, it turns out that the assumption $\|g(x^*)\| > 0$ is impossible too. $\square$

In the implementation of Algorithm 3.1, the choice of the trial point, which depends on the selection of t in the interval $[0.1, 0.9]$, is crucial. When the current point is not a forbidden point, the function value at that point is meaningful, and the new trial value t can be selected using a quadratic or cubic interpolation process with safeguards (given by 0.1 and 0.9 in our description, though other values could obviously be used). However, when the current point is a forbidden point, choosing the new trial point via interpolation makes no sense. In that case, one must resort to updates that do not rely on the function value at the discarded point. Typically, the new t is defined as the discarded one multiplied by either 0.1 or 0.5.

4 Regularization algorithms

The main algorithmic consequence of having to minimize a very expensive function is that any computational work carried out by the algorithm other than evaluating the function itself is considered insignificant. In this sense, we identify a drawback in Algorithm 3.1. Indeed, in that algorithm, when a new trial point is discarded, the new point is selected using procedures with negligible computational cost, such as one-dimensional interpolation or backtracking. However, when dealing with a very expensive function, one could employ relatively cheap yet much more complex and comprehensive models to compute the new trial points. For instance, one could minimize a high-order model to obtain the first trial point of an iteration and, if a trial point is rejected, minimize the same model over a smaller (trust) region or by adding a regularization term to it, which is essentially equivalent. It is along these lines that the alternative described in this section is developed.

{complexity}

Consider the problem (1) in which the objective function is a sum of squares:

$$f(x) = \sum_{i=1}^m f_i(x)^2 \quad (17) \quad \{\text{lasoma}\}$$

We assume that the functions f_i have continuous derivatives for all $x \notin P$.

Algorithm 4.1.

Let $x^0 \in \mathbb{R}^n - P$ be the initial approximation, $\alpha > 0$, $\sigma_{\min} > 0$, $M > 0$, $0 < \theta < 1$.

Step 0. Initialize $k \leftarrow 0$.

Step 1. Set $\sigma > 0$.

Step 2. Define, for all $i = 1, \dots, m$, $H_{k,i} \in \mathbb{R}^{n \times n}$ a symmetric matrix such that $\|H_{k,i}\| \leq M$.

Step 2. Solve, approximately,

$$\text{Minimize } \sum_{i=1}^m [f_i(x^k) + \nabla f_i(x^k)^T s + \frac{1}{2} s^T H_{k,i} s]^2 + \sigma \|s\|^2, \quad (18) \quad \{\text{regsub}\}$$

obtaining the trial step s^{trial} , that satisfies

$$\sum_{i=1}^m (f_i(x^k) + \nabla f_i(x^k)^T s^{trial} + \frac{1}{2} [s^{trial}]^T H_{k,i} s^{trial})^2 + \sigma \|s^{trial}\|^2 \leq f(x^k) \quad (19) \quad \{\text{bajomo}\}$$

and

$$\|\nabla_s \left(\sigma \|s^{trial}\|^2 + \sum_{i=1}^m (f_i(x^k) + \nabla f_i(x^k)^T s^{trial} + \frac{1}{2} [s^{trial}]^T H_{k,i} s^{trial})^2 \right)\| \leq \theta \|\nabla f(x^k)\| \quad (20) \quad \{\text{gcero}\}$$

Step 3. If

$$x^k + s^{trial} \notin P \quad (21) \quad \{\text{feasi}\}$$

and

$$f(x^k + s^{trial}) \leq f(x^k) - \alpha \|s^{trial}\|^2, \quad (22) \quad \{\text{armial}\}$$

define

$$s^k = s^{trial}, \quad (23) \quad \{\text{stsk}\}$$

$$x^{k+1} = x^k + s^k, \quad (24) \quad \{\text{novox}\}$$

set

$$k \leftarrow k + 1, \quad (25) \quad \{\text{next}\}$$

$$\sigma_k \leftarrow \min\{\sigma_{min}, \sigma/10\}, \quad (26) \quad \{\text{sigmak}\}$$

and go to Step 1.

Otherwise, set

$$\sigma \leftarrow 10\sigma \quad (27) \quad \{\text{upsig}\}$$

and go to Step 2.

Remark

Note that, in Step 1, the initial trial value of σ at iteration k is only required to be strictly greater than 0; it need not equal σ_{min} . The reason is that, although starting from $\sigma_0 = 1$ is a reasonable choice at iteration $k = 0$, in later iterations it is preferable to let this value depend on the σ actually accepted at the end of the previous iteration. For instance, one may start the new iteration with the same σ accepted previously, or with a fraction of it. The latter is the choice implemented throughout this work (Section 6), where the trial σ at the start of each iteration is the previously accepted value divided by 10.

Lemma 4.1 *Step 2 of Algorithm 4.1 is well defined.*

Proof: We need to show that s^{trial} satisfying (19) and (20) exist. Note that the value of the objective function of (18) at $s = 0$ is $f(x^k)$. Let us show that the level set of this function defined by $f(x^k)$ is bounded. In fact, if $\sigma \|s\|^2 > f(x^k)$ we have that the objective function of (18) is greater than $f(x^k)$. Therefore, the level set defined by $f(x^k)$ is strictly contained in the ball $\mathcal{B}(x^k, \sqrt{f(x^k)} + 1)$, which is a compact set. Therefore, by continuity, a minimizer in the ball $\mathcal{B}(x^k, \sqrt{f(x^k)} + 1)$ exists and, since this minimizer cannot be in the boundary, it turns out that the gradient of the objective function vanishes. Therefore, both (19) and (20) take place. $\square$

Remark

Standard unconstrained minimization algorithms applied to problem (18) and starting from $s^0 = 0$ necessarily find points that satisfy (19) and (20), as these algorithms converge monotonically, in some sense, to stationary points of the minimization problem.

Lemma 4.2 *If $\sigma > 0$ and $s \in \mathbb{R}^n$ is such that*

$$\sum_{i=1}^m (f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_{k,i} s)^2 + \sigma \|s\|^2 \leq f(x) \quad (28) \quad \{\text{bajom}\}$$

we have that

$$\|s\|^2 \leq f(x)/\sigma \quad (29) \quad \{\text{cotas}\}$$

Proof: By (28) and the fact that $\sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_{k,i} s]^2 \geq 0$ we have that $\sigma \|s\|^2 \leq f(x)$. $\square$

The following assumption holds when the gradients ∇f_i satisfy Lischitz conditions given by

$$\|\nabla f_i(x) - \nabla f_i(z)\| \leq \gamma \|x - z\| \text{ for all } x, z \in \mathbb{R} - P \text{ such that } \|x - z\| \leq \delta_{lips}. \quad (30) \quad \{\text{lipschitz}\}$$

Assumption 4.1

There exist $\gamma > 0$ and $\delta_{lips} > 0$ such that, for all $x, z \notin P$ such that $\|x - z\| \leq \delta_{lips}$, and $i = 1, \dots, m$,

$$f_i(z) \leq f_i(x) + \nabla f_i(x)^T (z - x) + \frac{\gamma}{2} \|z - x\|^2. \quad (31) \quad \{\text{taylor}\}$$

Theorem 4.1. *Suppose that Assumption 4.1 holds. Then Algorithm 4.1 is well defined.*

Moreover, if $\mathcal{B}(x^k, \delta) \subset \mathbb{R}^n - P$, defining $\underline{\delta} = \min\{\delta_{lips}, \delta\}$, we have that

$$\sigma_k \leq 10 \max \left\{ \frac{f(x^k)}{\underline{\delta}^2}, \alpha + m(M+1) \left[M + \frac{\gamma}{2} \right] \underline{\delta}^2 + [2M + \gamma] \sum_{i=1}^m [|f_i(x^k)| + \|\nabla f_i(x^k)\| \underline{\delta}] \right\}. \quad (32) \quad \{\text{sigk}\}$$

Proof: We need to prove that, for all x^k generated by Algorithm 4.1, the iterate x^{k+1} can be computed in finite time. Let us denote, for simplicity, $x = x^k$ and $H_i = H_{k,i}$.

By the definition of Algorithm 4.1, $x^0 \in \mathbb{R}^n - P$ is well defined. Assume, as inductive hypothesis, that $x^k \in \mathbb{R}^n - P$ is well defined. Since P is closed, there exists $\delta > 0$ such that $\mathcal{B}(x^k, \delta) \subset \mathbb{R}^n - P$.

By (29), a sufficient condition that guarantees that $x^k + s^{trial} \in \mathcal{B}(x^k, \min\{\delta_{lips}, \delta\})$ is $f(x^k)/\sigma \leq \min\{\delta_{lips}, \delta\}^2$. So,

$$\sigma \geq f(x^k)/\min\{\delta_{lips}, \delta\}^2 \Rightarrow x^k + s^{trial} \in \mathcal{B}(x^k, \min\{\delta_{lips}, \delta\}). \quad (33) \quad \{\text{simama}\}$$

Therefore, after a finite number of increases of σ , starting from $\sigma \geq \sigma_{min}$, we get the fulfillment of (21) and (22) or we get $\|s^{trial}\| \leq \min\{\delta_{lips}, \delta\}$. Furthermore, again by (29), if $\|s^{trial}\| \leq \min\{\delta_{lips}, \delta\}$ is rejected because (22) fails to hold, the next trials $x^k + s^{trial}$ also belong to $\mathcal{B}(x^k, \delta)$.

For simplicity, let us denote $s = s^{trial}$, $x = x^k$, and $H_i = H_{k,i}$.

By (31), for all $i = 1, \dots, m$, if $x + s \in \mathcal{B}(x, \min\{\delta_{lips}, \delta\})$, we have that

$$f_i(x + s) \leq f_i(x) + \nabla f_i(x)^T s + \frac{\gamma}{2} \|s\|^2$$

Then, since $\|H_i\| \leq M$,

$$f_i(x + s) \leq f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s + M \|s\|^2 + \frac{\gamma}{2} \|s\|^2$$

So,

$$f_i(x+s)^2 \leq [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s]^2 + \left[M + \frac{\gamma}{2}\right]^2 \|s\|^4 + 2[f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s] \left[M + \frac{\gamma}{2}\right] \|s\|^2.$$

Therefore,

$$\sum_{i=1}^m f_i(x+s)^2 \leq m \left[M + \frac{\gamma}{2}\right]^2 \|s\|^4 + \sum_{i=1}^m \left([f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s]^2 + 2[f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s] \left[M + \frac{\gamma}{2}\right] \|s\|^2 \right).$$

Now, by (19), we have that

$$\sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s]^2 \leq \sum_{i=1}^m f_i(x)^2 - \sigma \|s\|^2,$$

then

$$\sum_{i=1}^m f_i(x+s)^2 \leq \sum_{i=1}^m f_i(x)^2 - \sigma \|s\|^2 + m \left[M + \frac{\gamma}{2}\right]^2 \|s\|^4 + 2 \left[M + \frac{\gamma}{2}\right] \|s\|^2 \sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s]$$

Therefore, $\sum_{i=1}^m f_i(x+s)^2 \leq \sum_{i=1}^m f_i(x)^2 - \alpha \|s\|^2$ whenever

$$-\sigma \|s\|^2 + m \left[M + \frac{\gamma}{2}\right]^2 \|s\|^4 + 2 \left[M + \frac{\gamma}{2}\right] \|s\|^2 \sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s] \leq -\alpha \|s\|^2.$$

This inequality holds whenever

$$m \left[M + \frac{\gamma}{2}\right]^2 \|s\|^4 + 2 \left[M + \frac{\gamma}{2}\right] \|s\|^2 \sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s] \leq (\sigma - \alpha) \|s\|^2.$$

A sufficient condition of the last inequality is:

$$\sigma \geq \alpha + m \left[M + \frac{\gamma}{2}\right]^2 \|s\|^2 + 2 \left[M + \frac{\gamma}{2}\right] \sum_{i=1}^m [f_i(x) + \nabla f_i(x)^T s + \frac{1}{2} s^T H_i s]$$

Since $\|s\| = \|s^{trial}\| \leq \min\{\delta_{lips}, \delta\}$, a sufficient condition for this inequality is:

$$\sigma \geq \alpha + m(M+1) \left[M + \frac{\gamma}{2}\right]^2 \min\{\delta_{lips}, \delta\}^2 + [2M + \gamma] \sum_{i=1}^m [|f_i(x)| + \|\nabla f_i(x)\| \min\{\delta_{lips}, \delta\}]. \quad (34) \quad \{\textbf{sigmar}\}$$

Clearly, this condition holds after a finite number of increases of σ at (27). Therefore, in order to satisfy (21) and (22), in the worst case σ should be increased up to satisfy $x^k + s^{trial} \in \mathcal{B}(x^k, \delta)$ and, then, should be increased up to the fulfillment of (34). This completes the proof. $\square$

Remark. It must be warned that, in our case, the fact that the decrease can be obtained using a bounded number of function evaluations does not imply that the regularization parameters σ_k are bounded, independently of k . Indeed, we can say that, for obtaining $x^k + s^{trial}$ we only need a finite number of function evaluations, but we do not know how much the regularization parameter must be increased. This is a crucial difference between this complexity analysis and the one that is usual when forbidden points are not presented.

Theorem 4.2 Suppose that Assumption 4.1 holds. Let $\{x^k\}$ be a sequence generated by Algorithm 4.1. Then,

$$\lim_{k \rightarrow \infty} \|s^k\| = 0. \quad (35) \quad \{\text{scero}\}$$

Moreover, given $\varepsilon > 0$, the number of iterations such that $\|s^k\| > \varepsilon$ is not bigger than $f(x^0)\varepsilon^{-2}/\alpha$. In particular, there exists $k_0 \in \mathbb{N}$, which only depends on x^0 and α , such that, for all $k \geq k_0$ we have that $\|s^k\| \leq 1$.

Proof: We have already proved that, at each iteration of the algorithm, (22) holds. Therefore, for all $k \in \mathbb{N}$,

$$f(x^k) \leq f(x^0) - \alpha(\|s^0\|^2 + \dots + \|s^{k-1}\|^2). \quad (36) \quad \{\text{suma}\}$$

Since $f(x^k) \geq 0$ for all $k \in \mathbb{N}$, this implies (35).

Moreover, let N_k be the number indices such that $\|s^j\| > \varepsilon$ and $j \leq k$.

By (36) we have that, for all $k \in \mathbb{N}$,

$$f(x^k) \leq f(x^0) - \alpha N_k \varepsilon^2.$$

Since $f(x^k) \geq 0$, this implies that

$$\alpha N_k \varepsilon^2 \leq f(x^0)$$

for all $k \in \mathbb{N}$.

So, for all $k \in \mathbb{N}$,

$$N_k \leq \frac{f(x^0)}{\alpha} \varepsilon^{-2}.$$

So, the desired result is proved. $\square$

Assumption 4.2

There exists $c_g > 0$ such that for all $x \notin P$ and $f(x) \leq f(x^0)$ we have that

$$\|\nabla f_i(x)\| \leq c_g \quad (37) \quad \{\text{boundg}\}$$

for all $i = 1, \dots, m$.

Theorem 4.3 Suppose that Assumptions 4.1 and 4.2 hold. Let $\{x^k\}$ be a sequence generated by Algorithm 4.1.

Suppose that $\delta > 0$ and $\mathcal{B}(x^k, \delta) \subset \mathbb{R}^n - P$.

Then, there exists $k_0 = k_0(f(x^0), \alpha) \in \mathbb{N}$, such that, for all $k \geq k_0$,

$$\|\nabla f(x^k)\| \leq c\|s^k\|, \quad (38) \quad \{\text{gsk}\}$$

where c only depends on $\delta, \delta_{lips}, \gamma, \alpha, m, M, c_g$, and $f(x^0)$.

Proof: By (20), if $s = s^{trial}$ and writing, for simplicity, $H_i = H_{k,i}$, we have that

$$\begin{aligned} \theta \|\nabla f(x^k)\| &\geq \left\| \nabla_s \left(\sigma \|s\|^2 + \sum_{i=1}^m (f_i(x^k) + \nabla f_i(x^k)^T s + \frac{1}{2} s^T H_i s)^2 \right) \right\| \\ &= \left\| 2\sigma s + 2 \sum_{i=1}^m \left(f_i(x^k) + \nabla f_i(x^k)^T s + \frac{1}{2} s^T H_i s \right) (\nabla f_i(x^k) + H_i s) \right\| \end{aligned}$$

Let us write

$$v = 2\sigma s + 2 \sum_{i=1}^m f_i(x^k) H_i s + 2 \sum_{i=1}^m (\nabla f_i(x^k)^T s) \nabla f_i(x^k) + 2 \sum_{i=1}^m (\nabla f_i(x^k)^T s) H_i s + \sum_{i=1}^m (s^T H_i s) \nabla f_i(x^k) + \sum_{i=1}^m (s^T H_i s) H_i s. \quad (39) \quad \{\text{eta}\}$$

Then, as $\nabla f(x^k) = 2 \sum_{i=1}^m f_i(x^k) \nabla f_i(x^k)$, we have that

$$\theta \|\nabla f(x^k)\| \geq \|\nabla f(x^k) + v\|.$$

So,

$$\begin{aligned} \theta \|\nabla f(x^k)\| &\geq \|\nabla f(x^k)\| - \|v\|, \\ (\theta - 1) \|\nabla f(x^k)\| &\geq -\|v\|, \end{aligned}$$

and

$$(1 - \theta) \|\nabla f(x^k)\| \leq \|v\|.$$

Then,

$$\begin{aligned} (1 - \theta) \|\nabla f(x^k)\| &\leq 2\sigma \|s\| + 2M \sum_{i=1}^m |f_i(x^k)| \|s\| + 2mc_g^2 \|s\| + 2mc_g M \|s\|^2 + Mmc_g \|s\|^2 + mM^2 \|s\|^3 \\ &\leq 2\sigma \|s\| + 2M\sqrt{m}\sqrt{f(x^0)} \|s\| + 2mc_g^2 \|s\| + 2mc_g M \|s\|^2 + Mmc_g \|s\|^2 + mM^2 \|s\|^3. \end{aligned}$$

Therefore,

$$\|\nabla f(x^k)\| \leq \frac{2\sigma \|s\| + 2M\sqrt{m}\sqrt{f(x^0)} \|s\| + 2mc_g^2 \|s\| + 2mc_g M \|s\|^2 + Mmc_g \|s\|^2 + mM^2 \|s\|^3}{1 - \theta}.$$

Then, for all $k \in \mathbb{N}$,

$$\|\nabla f(x^k)\| \leq \frac{2\sigma \|s^{trial}\| + 2M\sqrt{m}\sqrt{f(x^0)} \|s^{trial}\| + 2mc_g^2 \|s^{trial}\| + 2mc_g M \|s^{trial}\|^2 + Mmc_g \|s^{trial}\|^2 + mM^2 \|s^{trial}\|^3}{1 - \theta}.$$

(Note that (40) has been obtained using only requirement (20) and is independent of whether s^{trial} is an accepted increment or not. Moreover, (40) holds even if $x^k + s^{trial}$ is a forbidden point.)

In particular, when $s^{trial} = s^k$, (40) yields

$$\|\nabla f(x^k)\| \leq \|s^k\| \left\{ \frac{2\sigma_k + 2M\sqrt{m}\sqrt{f(x^0)} + 2mc_g^2 + 2mc_g M \|s^k\| + Mmc_g \|s^k\| + mM^2 \|s^k\|^2}{1 - \theta} \right\}. \quad (41) \quad \{\text{gracr}\}$$

By Theorem 4.2, the number of iterations at which $\|s^k\| > 1$ is not bigger than $f(x^0)/\alpha$. So, there exists $k_0 \in \mathbb{N}$ such that $\|s^k\| \leq 1$ for all $k \geq k_0$. Then, if $k \geq k_0$ we have that

$$\|\nabla f(x^k)\| \leq \|s^k\| \left\{ \frac{2\sigma_k + 2M\sqrt{m}\sqrt{f(x^0)} + 2mc_g^2 + 2mc_g M + Mmc_g + mM^2}{1 - \theta} \right\}. \quad (42) \quad \{\text{gric}\}$$

So, if $k \geq k_0$ and $\sigma_k = 0$ we have that

$$\|\nabla f(x^k)\| \leq \|s^k\| \left\{ \frac{2M\sqrt{m}\sqrt{f(x^0)} + 2mc_g^2 + 2mc_g M + Mmc_g + mM^2}{1 - \theta} \right\}. \quad (43) \quad \{\text{groc}\}$$

Now, we need to prove that σ_k only depends on δ . By (32), using that $f(x^k) \leq f(x^0)$ and that $\sum_{i=1}^m |f_i(x^k)| \leq \sqrt{f(x^k)}\sqrt{m}$ we have that

$$\sigma_k \leq 10 \max \left\{ \frac{f(x^0)}{\min\{\delta_{lips}, \delta\}^2}, \alpha + m(M + 1) \left[M + \frac{\gamma}{2} \right] \min\{\delta_{lips}, \delta\}^2 + [2M + \gamma] [\sqrt{mf(x^0)} + \sum_{i=1}^m \|\nabla f_i(x^k)\| \min\{\delta_{lips}, \delta\}] \right\}.$$

So, by Assumption 4.2,

$$\sigma_k \leq 10 \max \left\{ \frac{f(x^0)}{\min\{\delta_{lips}, \delta\}^2}, \alpha + m(M+1) \left[M + \frac{\gamma}{2} \right] \min\{\delta_{lips}, \delta\}^2 + [2M + \gamma] [\sqrt{mf(x^0)} + mc_g \min\{\delta_{lips}, \delta\}] \right\}. \quad (44) \quad \{\text{signo}\}$$

By (42), (43) and (44) we have that, for all $k \geq k_0$, we have that

$$\|\nabla f(x^k)\| \leq c\|s^k\|,$$

where c only depends of $\delta, \delta_{lips}, \alpha, m, M, \gamma, c_g,$ and $f(x^0)$. This completes the proof. $\square$

Theorem 4.4 *Suppose that Assumptions 4.1 and 4.2 hold. Let $\{x^k\}$ be a sequence generated by Algorithm 4.1. Assume that $\varepsilon > 0$ and $\delta > 0$. Then, the number of iterations at which*

$$\mathcal{B}(x^k, \delta) \subset \mathbb{R}^n - P \text{ and } \|\nabla f(x^k)\| > \varepsilon \quad (45) \quad \{\text{epde}\}$$

is smaller than $k_0(f(x^0), \alpha) + \frac{cf(x^0)}{\alpha}\varepsilon^{-2}$ where $k_0(f(x^0), \alpha)$ and c are as in Theorem 4.3.

Proof: The proof follows straightforwardly from Theorems 4.2 and 4.3. $\square$

Corollary 4.1 *Suppose that Assumptions 4.1 and 4.2 hold. Let $\{x^k\}$ be a sequence generated by Algorithm 4.1. Suppose that there exists $\delta > 0$ such that for infinitely many iterations $\{x^k\}_{k \in K}$ we have that $\mathcal{B}(x^k, \delta) \subset \mathbb{R}^n - P$. Then,*

$$\lim_{k \in K} \|\nabla f(x^k)\| = 0.$$

Proof: Let $\varepsilon > 0$. By Theorem 4.4 the number of iterations at which (45) holds is finite. Then, there exists k_1 such that for $k \in K, k \geq k_1$ one has that $\|\nabla f(x^k)\| \leq \varepsilon$. Since ε is arbitrary this implies the thesis of this theorem.

5 The degeneracy at $k = 0$

Algorithm 4.1 minimizes $f(x) = \frac{1}{2}\|F(x)\|^2$, where $F = (f_1, \dots, f_m)^\top$ is the residual vector. At external iteration k it keeps one symmetric matrix H_i^k per residual component f_i , approximating $\nabla^2 f_i(x^k)$, and solves

$$M_k(s) = \sigma\|s\|^2 + \frac{1}{2} \sum_i q_i(s)^2, \quad q_i(s) = f_i(x^k) + \nabla f_i(x^k)^\top s + \frac{1}{2} s^\top H_i^k s, \quad (46) \quad \{\text{regsub_new}\}$$

accepting or rejecting $x^k + s$ by the ratio between actual and predicted reduction, and growing/shrinking σ accordingly. At $k = 0$, every $H_i^0 = 0$ (no secant pair has been observed yet), so (46) collapses to

$$M_0(s) = \sigma\|s\|^2 + \frac{1}{2} \sum_i (f_i(x^0) + \nabla f_i(x^0)^\top s)^2, \quad (47) \quad \{\text{eq:gn}\}$$

a single damped Gauss–Newton subproblem. Two things are worth noting about this particular instance of the subproblem. Unlike every subsequent iteration, $\sigma = \sigma_0 = 1$ here is a fixed constant, not yet informed by any accept/reject test on this problem, so how close (47)’s solution already sits to the scaled steepest-descent direction it approaches as $\sigma \rightarrow \infty$ (where the minimizer of (47) is asymptotically parallel to $-\nabla f(x^0)$) is essentially a guess about the local curvature scale of f at x^0 that the method has had no chance to calibrate yet. Moreover, there is, by construction, no curvature information at all to justify solving a quartic (rather than a quadratic) model here. The quartic term $\frac{1}{2} s^\top H_i^k s$ is identically zero for every i . So the very first step is the one iteration where the method’s own machinery (quasi-Newton curvature, calibrated regularization) has nothing to work with.

5.1 The closed form of M_0

{sec:closedform}

Rather than appealing to a different algorithm, the cleanest justification comes directly out of (47) itself, which is an unconstrained, strictly convex quadratic in s and can be solved in closed form. Write $J := F'(x^0)$ for the Jacobian of F at x^0 (the matrix whose i -th row is $\nabla f_i(x^0)^\top$), so that $g := \nabla f(x^0) = J^\top F(x^0)$, and $A := J^\top J$ (symmetric positive semidefinite). Then

$$\nabla_s M_0(s) = (2\sigma I + A)s + g = 0 \implies s(\sigma) = -(A + 2\sigma I)^{-1}g.$$

Lemma 5.1.1 (Descent for every $\sigma > 0$). *For every $\sigma > 0$ and every $g \neq 0$, $s(\sigma)$ is a descent direction for f at x^0 , satisfying $g^\top s(\sigma) < 0$.*

Proof. Since $A + 2\sigma I \succ 0$ for $\sigma > 0$, its inverse is symmetric positive definite, hence

$$g^\top s(\sigma) = -g^\top (A + 2\sigma I)^{-1}g < 0 \quad \text{whenever } g \neq 0.$$

This completes the proof. $\square$

So $s(\sigma)$ is a genuine descent direction for f for every $\sigma > 0$, not just asymptotically. The σ -loop never has a “wrong-direction” problem at $k = 0$. What it can have is a badly sized step, which is what the accept/reject test and the growth of σ are actually correcting for.

Proposition 5.1.2 (Alignment with $-g$ as $\sigma \rightarrow \infty$). *As $\sigma \rightarrow \infty$,*

$$s(\sigma) = -\frac{1}{2\sigma}g + o\left(\frac{1}{\sigma}\right) \implies \frac{s(\sigma)}{\|s(\sigma)\|} \longrightarrow -\frac{g}{\|g\|}. \quad (48) \quad \{\text{eq:asymptotic}\}$$

Proof. Factor 2σ out of the inverse:

$$s(\sigma) = -(A + 2\sigma I)^{-1}g = -\frac{1}{2\sigma} \left(I + \frac{A}{2\sigma} \right)^{-1} g.$$

As $\sigma \rightarrow \infty$, $A/(2\sigma) \rightarrow 0$, and matrix inversion is continuous at the identity, so $(I + A/(2\sigma))^{-1} \rightarrow I$, which gives the stated expansion. This completes the proof. $\square$

In words, for σ large enough, the solution of the degenerate $k = 0$ subproblem (47) already is, to leading order, a steepest-descent step of length $\approx 1/2\sigma$. The two are asymptotically the same computation. This is a direct consequence of (47)’s own closed form, entirely internal to Algorithm 4.1.

In practice, the rejection ladder $\sigma_0, \sigma_{j+1} = \gamma\sigma_j$ ($\gamma > 1$ the growth factor) tries $s(\sigma_0), s(\sigma_1), \dots$ until one is accepted. By Proposition 5.1.2, once $\sigma_j \gg \|A\|/2$ the successive trials are already essentially parallel to $-g$ and shrink by the fixed factor $1/\gamma$, so once it reaches that scale, the ladder is already behaving like a backtracking line search along $-g$. This is also a reason not to let σ_0 run away unchecked. Because σ is only shrunk, never reset, between outer iterations, an unusually large σ_0 , forced whenever the true first step happens to be small, persists into σ_1 and can over-dampen the next iteration’s subproblem as well.

5.2 Why consider this scheme

Solving a coefficient-estimation problem for a PDE is expensive. Evaluating the objective function requires solving the PDE once, and differentiating that objective requires differentiating the PDE solver itself. In [19] we solved this problem without derivatives, but the resulting computational cost was prohibitively high. In this work we instead use derivatives obtained by automatic differentiation.

This approach computes an exact derivative of the PDE solver, rather than a finite-difference approximation, via forward-mode automatic differentiation, using Julia’s `ForwardDiff.jl`. We chose forward mode over reverse (backward) mode because reverse-mode differentiation must first record every elementary operation performed by the solver onto a computational tape before it can evaluate the derivative, and since the PDE solver performs a very large number of operations, this tape would require a prohibitive amount of memory, on the order of 3 gigabytes for a single derivative evaluation in our setting.

Using as much derivative information as possible is valuable in principle, but Figure 1 shows why this is impractical here. The cost of computing the exact Hessian by automatic differentiation grows far faster with the problem dimension than the cost of the gradient, so exact second-order information should be avoided, relying instead on first-order (gradient) information alone.

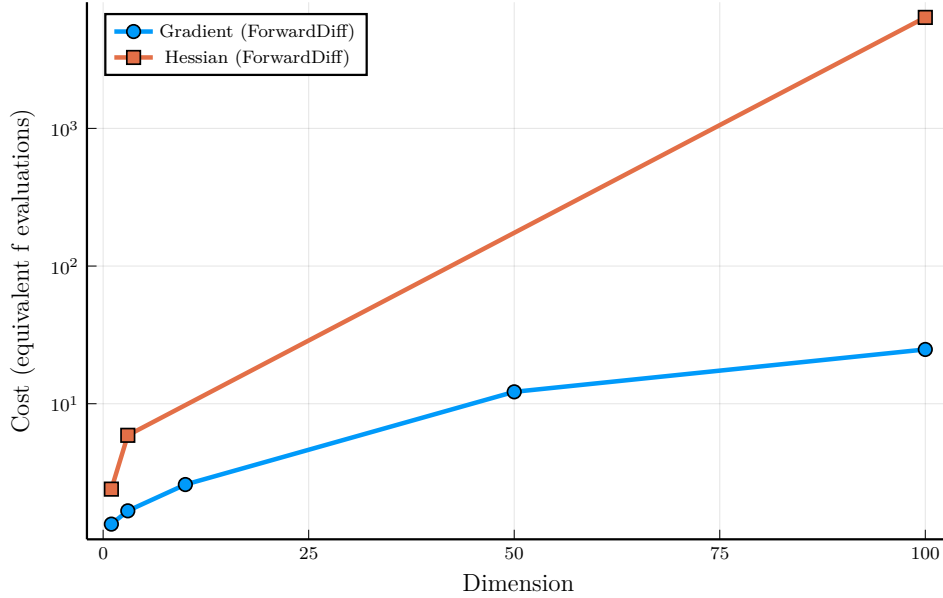


Figure 1: Cost, in equivalent evaluations of f , of computing the gradient and the Hessian of the PDE solver’s objective by automatic differentiation, as a function of the problem dimension.

{fig:tempo_deriv}

At dimension 100, Figure 1 shows that a single gradient evaluation already costs about as much as 25 evaluations of f , while a single Hessian evaluation costs more than 6400 evaluations of f .

For this reason, we need to make full use of the information already available to us, particularly the gradient, in order to solve this problem efficiently.

6 Computational tests

{sec:computation}

In this section we evaluate the performance of Algorithm 4.1 and compare it against a classical sum-of-squares approach. We run two types of tests. In the first, we consider the MGH (Moré, Garbow, and Hillstom) sum-of-squares test problems [31], a benchmark of 34 classical problems. This part is important to us because it helps us settle on a good solver and Hessian approximation. In the second part, we solve a hydraulic problem in which we need to estimate a coefficient of a PDE called the Saint-Venant equation.

6.1 Algorithm 4.1 for MGH problems

{sec:mgh-problem}

Algorithm 4.1 minimizes $f(x) = \frac{1}{2}\|F(x)\|_2^2$ by minimizing, at each iteration, a local quartic model of the residuals in place of a line search. Following Algorithm 4.1 (Eq. (21)), the step s at iteration k solves

$$M_k(s) = \sigma\|s\|_2^2 + \frac{1}{2} \sum_{i=1}^m q_i(s)^2, \quad q_i(s) = f_i(x_k) + \nabla f_i(x_k)^\top s + \frac{1}{2} s^\top H_i s,$$

where f_i and ∇f_i are the i -th residual and its gradient at x_k (Section 3), H_i is the current quasi-Newton approximation to its Hessian. Four further variants are compared in Table 1, and $\sigma\|s\|_2^2$ is the Levenberg-Marquardt-style regularization of Algorithm 4.1 that damps the step in place of a line search. The candidate $x_k + s$ is checked for feasibility as in Step 3 (Eq. 21). Acceptance then uses the ratio ρ_k between the actual and the model-predicted reduction in f , a refinement of Algorithm 4.1’s simpler decrease test (Eq. (25)), accepting whenever $\rho_k \geq \eta_1$ for a fixed threshold η_1 .

The implementation tested below uses the regularization update of Eq. (27) with the following parameters. The first iteration starts at $\sigma = 1$. Each accepted step divides σ by a fixed factor $\sigma_{grow} = 10$, and each rejected step, as in Eq. (30), multiplies σ by the same factor $\sigma_{grow} = 10$ and re-solves the subproblem at the same point, until a step is accepted. Between outer iterations, σ always carries over from the previous one rather than being reset. The floor $\sigma_{min} = 10^{-8}$ and the ceiling $\sigma_{max} = 10^{12}$ are used purely for numerical stability, preventing σ from underflowing or overflowing over the course of a run.

Every run reported below stops under one of three point-wise criteria, evaluated with the same default tolerances for Algorithm 4.1 and for Optim.jl’s BFGS [30] (kept equal so the comparison is fair):

- **Gradient:** the gradient norm reaches $\|\nabla f\| \leq 10^{-8}$. This is a genuine first-order stationary point (reported as `grad_conv` for both Algorithm 4.1 and BFGS in the tables below).
- **Step:** the last step has (numerically) zero length. The solver stalls and stops making progress (reported as `step_conv` for both Algorithm 4.1 and BFGS).
- **Function:** the objective value does not change between two consecutive iterations. The iterate is stuck on a plateau (reported as `func_conv` for both Algorithm 4.1 and BFGS).
- **Stalled:** Algorithm 4.1’s regularization parameter exceeded its ceiling, that is, $\sigma > 10^{12}$ (reported as `reg_stalled`. BFGS has no equivalent criterion).

Gradient, Step, and Function use the same name for both methods by construction: BFGS’s own status is classified by checking `Optim.g_converged`, then `Optim.x_converged`, then `Optim.f_converged` on its result, in that order, so the two methods can be compared under one shared vocabulary instead of each reporting its own native termination code.

Table 1 compares the six Algorithm 4.1 Hessian-update models, each run independently on the 34 MGH least-squares problems [31]. Its “Converged (gradient)” column counts how many of the 34 problems stopped under the gradient criterion. The remaining columns are mean $\pm$ standard deviation over the 34 problems. Each model keeps its own set of matrices $H_i \approx \nabla^2 f_i(x_k)$, $i = 1, \dots, m$, updated at every accepted step from $s = x_{k+1} - x_k$ and $y_i = \nabla f_i(x_{k+1}) - \nabla f_i(x_k)$:

- **BFGS:** the classical rank-two secant update [9, 18, 21, 40, 33] (Eq. (19)),

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i}.$$

- **SR1:** the symmetric rank-one update [33] (Eq. (20)),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)(y_i - H_i s)^T}{(y_i - H_i s)^T s}.$$

- **DW** (Dennis-Wolkowicz [14], Theorem 4.5(iii)): a Broyden-class update that adds a third, symmetrizing rank-one term to the BFGS pair,

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i} + (s^T y_i) w_i w_i^T, \quad w_i = \frac{y_i}{s^T y_i} - \frac{H_i s}{s^T H_i s}.$$

- **PSB** (Powell-Symmetric-Broyden [35]): a rank-two update that does not preserve positive definiteness (like SR1),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s) s^T + s (y_i - H_i s)^T}{s^T s} - \frac{s^T (y_i - H_i s)}{(s^T s)^2} s s^T.$$

Table 1: Comparison of the Algorithm 4.1 Hessian-update models on the 34 MGH problems.

Model	Converged (gradient)	Iterations	Func. evals	Grad. evals	Time (s)
BFGS	29/34	56.88 ± 188.25	99.97 ± 351.85	57.88 ± 188.25	2.071 ± 6.121
SR1	32/34	23.12 ± 54.84	44.32 ± 112.92	24.12 ± 54.84	0.934 ± 2.992
DW	29/34	56.12 ± 181.91	103.24 ± 351.45	57.12 ± 181.91	1.865 ± 5.942
PSB	31/34	15.68 ± 22.53	31.38 ± 53.44	16.68 ± 22.53	0.424 ± 0.919

{tab:hessian-mod}

PSB and SR1 are the two best models. Together they solve the most problems under the gradient criterion, and PSB in particular does so with by far the fewest iterations, evaluations, and execution time on average (Table 1). SR1 nonetheless certifies gradient convergence on one more problem than PSB (32/34 against 31/34). To see exactly where they disagree, Table 2 isolates the three problems on which at least one of the two methods does not stop under the gradient criterion.

Table 2: Algorithm 4.1 (PSB) vs. Algorithm 4.1 (SR1): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH10	Algorithm 4.1 (PSB)	4.397 293×10 ¹	5.754 922×10 ⁻⁵	299	127	step.conv
MGH10	Algorithm 4.1 (SR1)	4.397 293×10 ¹	5.500 886×10 ⁻²	297	125	step.conv
MGH16	Algorithm 4.1 (PSB)	4.291 110×10 ⁴	9.258 506×10 ⁻⁵	98	36	func.conv
MGH16	Algorithm 4.1 (SR1)	4.291 110×10 ⁴	6.255 963×10 ⁻⁵	44	9	func.conv
MGH31	Algorithm 4.1 (PSB)	1.528 639	2.527 165×10 ⁻⁸	62	21	func.conv
MGH31	Algorithm 4.1 (SR1)	1.528 639	4.387 102×10 ⁻⁹	44	29	grad.conv

{tab:psb-vs-sr1}

On MGH10 and MGH16, neither model reaches the gradient criterion, both stall under the step and function criteria, respectively, at essentially the same objective value, although Algorithm 4.1 with PSB reaches a smaller gradient norm on MGH10. For this reason, PSB is the best choice between the two approaches when weighing cost against benefit.

Table 3 repeats the comparison, but now running Algorithm 4.1 in full with PSB model on the same 34 problems. Newton/BFGS/BOBYQA solve only the small quartic model at each outer iteration, exactly as Algorithm 4.1 uses them, instead of standing in as the whole optimizer. “Newton” here is the exact Newton step on the quartic subproblem, computed via the Ipopt interior-point solver.

Function and gradient evaluations here count only the calls to the real residual and Jacobian made by Algorithm 4.1’s outer loop. The internal solver’s own work on the quartic surrogate is tracked separately, since evaluating the true function and its derivative is considerably more expensive than evaluating the model. Because we are solving a quartic (rather than a quadratic) problem, we use a multistart method to search for a global optimum. At each outer iteration k , 100 random points are drawn from a ball $B(0, \max\{1, d_{k-1}\})$. Using the default multistart setting for every method would be impractical across all 34 problems in this table.

Table 3: Algorithm 4.1 (PSB), varying only `model_solver`, on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Algorithm 4.1 (PSB, Newton)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.12 ± 21.90	30.21 ± 49.59	16.12 ± 21.90
Algorithm 4.1 (PSB, BFGS)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.26 ± 21.99	30.26 ± 48.63	16.26 ± 21.99
Algorithm 4.1 (PSB, BOBYQA)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	30/34	39.74 ± 108.86	57.68 ± 127.48	40.74 ± 108.86

Newton and BFGS are essentially indistinguishable (31/34, within 1% of each other on every column), and even BOBYQA, which needed an order of magnitude more evaluations and converged on only 10/34 when applied directly to the raw residual, falls behind by only one problem (30/34) and roughly $2.6\times$ more outer iterations on average. This distinction matters because evaluating the derivative can sometimes cost more than $2.6\times$ what evaluating the objective function costs. Since Newton achieves the best performance among the three, we adopt it as the model solver used throughout the rest of this work.

6.2 Sum of square problem BFGS configuration

We now consider a different configuration than the average BFGS method. We do this because the default line search, generally the Hager-Zhang line search, evaluates the derivative of the function many times. For this reason, we now test different classical line searches for the BFGS method. Table 4 checks some line searches for `Optim.BFGS` on the same 34 MGH problems with four line searches. Hager and Zhang (`Optim.jl`'s default), a minimalist backtracking with Armijo condition, and Backtracking with quadratic (order 2) and cubic (order 3) interpolation. “Converged (gradient)” counts how many of the 34 problems stopped under the gradient criterion. The remaining columns are mean $\pm$ standard deviation over all 34 problems (including the ones that stopped under a different criterion).

Table 4: Comparison of line searches for `Optim.jl`'s BFGS on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Hager-Zhang	$1.29 \times 10^3 \pm 7.36 \times 10^3$	33/34	66.91 ± 120.92	107.82 ± 206.62	107.82 ± 206.62
Backtracking	$2.49 \times 10^7 \pm 1.45 \times 10^8$	30/34	51.56 ± 93.74	89.62 ± 131.90	52.47 ± 93.75
Backtracking (order 2)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	32/34	61.59 ± 99.26	86.74 ± 134.56	62.56 ± 99.17
Backtracking (order 3)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	56.91 ± 89.47	84.91 ± 132.01	57.85 ± 89.33

Hager-Zhang converges by gradient on the most problems (33/34) but at the highest average cost. Backtracking is a cheap line search, but its value of f and its convergence rate per problem are not as good on average (30/34). Backtracking order 2 and order 3 have similar results. Although Backtracking order 3 has slightly fewer evaluations, Backtracking order 2 found gradient convergence on 32/34 problems. For this reason, we choose Backtracking order 2 to compare with our approach.

6.3 Algorithm 4.1 against BFGS with backtracking

Algorithm 4.1 (PSB) uses persistent decaying σ , a full Hessian reset on a bad ratio, and $-\nabla f$ +Armijo at $k = 0$ and as a safeguard. The following table compares it against BFGS with backtracking line search on the 34 MGH least-squares problems.

Table 5: Algorithm 4.1 (PSB) vs. BFGS (backtracking): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH01	Algorithm 4.1 (PSB)	2.415715×10^{-23}	8.22188×10^{-12}	12	8	grad_conv
MGH01	BFGS (backtracking)	1.981121×10^{-23}	3.687403×10^{-10}	53	40	grad_conv
MGH02	Algorithm 4.1 (PSB)	5.096567×10^{-18}	3.844849×10^{-9}	10	7	grad_conv

continued on next page

Table 5: (continued)

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH02	BFGS (backtracking)	$1.106\,930 \times 10^{-26}$	$2.198\,151 \times 10^{-12}$	15	11	grad_conv
MGH03	Algorithm 4.1 (PSB)	$1.576\,836 \times 10^{-12}$	$2.568\,219 \times 10^{-9}$	31	20	grad_conv
MGH03	BFGS (backtracking)	$5.053\,640 \times 10^{-32}$	$2.830\,752 \times 10^{-10}$	233	174	grad_conv
MGH04	Algorithm 4.1 (PSB)	$3.811\,648 \times 10^{-18}$	$7.733\,972 \times 10^{-9}$	10	8	grad_conv
MGH04	BFGS (backtracking)	$9.860\,761 \times 10^{-34}$	$4.440\,892 \times 10^{-10}$	40	14	grad_conv
MGH05	Algorithm 4.1 (PSB)	$1.307\,511 \times 10^{-16}$	$6.948\,485 \times 10^{-9}$	10	8	grad_conv
MGH05	BFGS (backtracking)	$2.500\,895 \times 10^{-24}$	$1.079\,551 \times 10^{-10}$	22	19	grad_conv
MGH06	Algorithm 4.1 (PSB)	$1.010\,000 \times 10^3$	$1.161\,571 \times 10^{-28}$	11	2	grad_conv
MGH06	BFGS (backtracking)	$1.010\,000 \times 10^3$	$1.161\,571 \times 10^{-28}$	10	2	grad_conv
MGH07	Algorithm 4.1 (PSB)	$1.492\,293 \times 10^{-16}$	$6.034\,835 \times 10^{-9}$	17	10	grad_conv
MGH07	BFGS (backtracking)	$5.236\,320 \times 10^{-18}$	$3.028\,697 \times 10^{-9}$	41	30	grad_conv
MGH08	Algorithm 4.1 (PSB)	$4.107\,439 \times 10^{-34}$	$4.903\,415 \times 10^{-10}$	12	9	grad_conv
MGH08	BFGS (backtracking)	$4.107\,439 \times 10^{-37}$	$3.394\,161 \times 10^{-10}$	34	28	grad_conv
MGH09	Algorithm 4.1 (PSB)	$5.639\,664 \times 10^{-97}$	$5.549\,518 \times 10^{-11}$	9	7	grad_conv
MGH09	BFGS (backtracking)	$5.639\,664 \times 10^{-91}$	$1.552\,131 \times 10^{-9}$	6	5	grad_conv
MGH10	Algorithm 4.1 (PSB)	$4.397\,293 \times 10^1$	$1.204\,293 \times 10^{-5}$	276	122	reg_stalled
MGH10	BFGS (backtracking)	$4.397\,293 \times 10^1$	2.281 500	552	386	step_conv
MGH11	Algorithm 4.1 (PSB)	$5.960\,004 \times 10^{-23}$	$3.251\,504 \times 10^{-13}$	26	17	grad_conv
MGH11	BFGS (backtracking)	$2.268\,388 \times 10^{-29}$	$7.169\,397 \times 10^{-10}$	50	42	grad_conv
MGH12	Algorithm 4.1 (PSB)	$3.799\,377 \times 10^{-18}$	$8.814\,938 \times 10^{-10}$	14	11	grad_conv
MGH12	BFGS (backtracking)	$1.579\,764 \times 10^{-23}$	$3.202\,002 \times 10^{-11}$	53	41	grad_conv
MGH13	Algorithm 4.1 (PSB)	$5.156\,722 \times 10^{-19}$	$6.613\,609 \times 10^{-9}$	12	9	grad_conv
MGH13	BFGS (backtracking)	$1.566\,490 \times 10^{-13}$	$3.607\,786 \times 10^{-9}$	55	45	grad_conv
MGH14	Algorithm 4.1 (PSB)	$9.424\,359 \times 10^{-18}$	$8.234\,967 \times 10^{-10}$	11	7	grad_conv
MGH14	BFGS (backtracking)	$3.521\,535 \times 10^{-25}$	$3.460\,933 \times 10^{-10}$	44	31	grad_conv
MGH15	Algorithm 4.1 (PSB)	$1.537\,528 \times 10^{-49}$	$9.059\,854 \times 10^{-9}$	38	22	grad_conv
MGH15	BFGS (backtracking)	$1.537\,528 \times 10^{-44}$	$4.132\,861 \times 10^{-10}$	39	33	grad_conv
MGH16	Algorithm 4.1 (PSB)	$4.291\,110 \times 10^4$	$6.530\,587 \times 10^{-5}$	91	37	reg_stalled
MGH16	BFGS (backtracking)	$4.291\,110 \times 10^4$	$2.831\,466 \times 10^{-5}$	72	26	func_conv
MGH17	Algorithm 4.1 (PSB)	$2.732\,447 \times 10^{-58}$	$8.377\,671 \times 10^{-9}$	24	12	grad_conv
MGH17	BFGS (backtracking)	$2.732\,447 \times 10^{-58}$	$8.227\,669 \times 10^{-9}$	90	59	grad_conv
MGH18	Algorithm 4.1 (PSB)	$1.002\,042 \times 10^{-18}$	$1.650\,363 \times 10^{-11}$	58	34	grad_conv
MGH18	BFGS (backtracking)	$2.827\,825 \times 10^{-31}$	$1.276\,728 \times 10^{-8}$	44	41	grad_conv
MGH19	Algorithm 4.1 (PSB)	$4.383\,556 \times 10^{-29}$	$1.198\,492 \times 10^{-9}$	115	59	grad_conv
MGH19	BFGS (backtracking)	$1.373\,773 \times 10^{-18}$	$8.291\,793 \times 10^{-10}$	147	113	grad_conv
MGH20	Algorithm 4.1 (PSB)	$1.143\,835 \times 10^{-31}$	$1.841\,972 \times 10^{-9}$	13	10	grad_conv
MGH20	BFGS (backtracking)	$1.143\,835 \times 10^{-37}$	$7.070\,265 \times 10^{-9}$	49	39	grad_conv
MGH21	Algorithm 4.1 (PSB)	$2.415\,715 \times 10^{-23}$	$3.106\,049 \times 10^{-11}$	12	8	grad_conv
MGH21	BFGS (backtracking)	$4.792\,661 \times 10^{-18}$	$1.495\,834 \times 10^{-9}$	228	150	grad_conv
MGH22	Algorithm 4.1 (PSB)	$4.516\,652 \times 10^{-14}$	$2.795\,657 \times 10^{-10}$	13	10	grad_conv
MGH22	BFGS (backtracking)	$1.653\,367 \times 10^{-13}$	$1.561\,713 \times 10^{-8}$	136	112	grad_conv
MGH23	Algorithm 4.1 (PSB)	$1.124\,989 \times 10^{-52}$	$2.279\,978 \times 10^{-9}$	13	10	grad_conv
MGH23	BFGS (backtracking)	$1.124\,989 \times 10^{-51}$	$2.205\,331 \times 10^{-8}$	87	68	grad_conv
MGH24	Algorithm 4.1 (PSB)	$1.728\,411 \times 10^{-61}$	$1.740\,229 \times 10^{-9}$	37	22	grad_conv
MGH24	BFGS (backtracking)	$1.728\,411 \times 10^{-66}$	$6.288\,297 \times 10^{-9}$	588	450	grad_conv
MGH25	Algorithm 4.1 (PSB)	$7.395\,571 \times 10^{-33}$	$3.845\,925 \times 10^{-16}$	13	6	grad_conv
MGH25	BFGS (backtracking)	$6.843\,630 \times 10^{-29}$	$7.268\,618 \times 10^{-12}$	23	16	grad_conv
MGH26	Algorithm 4.1 (PSB)	$1.397\,528 \times 10^{-52}$	$2.997\,834 \times 10^{-9}$	27	16	grad_conv

continued on next page

Table 5: (continued)

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH26	BFGS (backtracking)	$1.397\,528 \times 10^{-5}$	$1.106\,362 \times 10^{-8}$	36	34	grad_conv
MGH27	Algorithm 4.1 (PSB)	$2.240\,730 \times 10^{-16}$	$9.941\,231 \times 10^{-9}$	11	8	grad_conv
MGH27	BFGS (backtracking)	$5.734\,697 \times 10^{-28}$	$3.689\,878 \times 10^{-9}$	20	17	grad_conv
MGH28	Algorithm 4.1 (PSB)	$2.223\,149 \times 10^{-16}$	$6.931\,523 \times 10^{-9}$	10	8	grad_conv
MGH28	BFGS (backtracking)	$2.533\,828 \times 10^{-19}$	$9.086\,349 \times 10^{-8}$	26	18	grad_conv
MGH29	Algorithm 4.1 (PSB)	$1.130\,540 \times 10^{-22}$	$9.906\,829 \times 10^{-11}$	8	7	grad_conv
MGH29	BFGS (backtracking)	$9.104\,593 \times 10^{-19}$	$9.563\,296 \times 10^{-9}$	8	8	grad_conv
MGH30	Algorithm 4.1 (PSB)	$7.642\,460 \times 10^{-23}$	$3.456\,063 \times 10^{-13}$	10	7	grad_conv
MGH30	BFGS (backtracking)	$1.965\,544 \times 10^{-19}$	$9.026\,641 \times 10^{-9}$	46	23	grad_conv
MGH31	Algorithm 4.1 (PSB)	1.528 639	$4.173\,205 \times 10^{-8}$	49	18	reg_stalled
MGH31	BFGS (backtracking)	1.340 110	$4.482\,312 \times 10^{-9}$	78	43	grad_conv
MGH32	Algorithm 4.1 (PSB)	5.000 000	$3.845\,925 \times 10^{-16}$	3	2	grad_conv
MGH32	BFGS (backtracking)	5.000 000	$5.578\,802 \times 10^{-16}$	2	2	grad_conv
MGH33	Algorithm 4.1 (PSB)	2.317 073	$6.326\,298 \times 10^{-11}$	12	5	grad_conv
MGH33	BFGS (backtracking)	2.317 073	$1.423\,632 \times 10^{-10}$	13	5	grad_conv
MGH34	Algorithm 4.1 (PSB)	3.067 568	$1.552\,202 \times 10^{-10}$	9	2	grad_conv
MGH34	BFGS (backtracking)	3.067 568	$2.134\,352 \times 10^{-10}$	9	2	grad_conv

Across the 34 problems, the two methods usually agree closely on the final objective value, but differ sharply in cost. Algorithm 4.1 (PSB) needs far fewer function and gradient evaluations than BFGS on most problems (see Table 5). MGH10 shows how hard some of these problems are. Neither method certifies gradient convergence there, and both land on essentially the same objective value ($f \approx 43.97$), yet Algorithm 4.1 still reaches a far smaller gradient norm than BFGS ($\|\nabla f\| \approx 1.2 \times 10^{-5}$ against ≈ 2.28), using half the function evaluations and under a third of the gradient evaluations. MGH18 shows the opposite kind of gap. Although both methods stop under the gradient criterion, Algorithm 4.1 reaches essentially zero ($f \approx 1.0 \times 10^{-18}$) while BFGS settles for a distinctly larger value ($f \approx 2.8 \times 10^{-3}$). A small gradient norm does not always mean the same stationary point.

The boxplot below compares the two approaches directly and uses the Mann-Whitney U test [27] to check whether the difference between them is statistically significant.

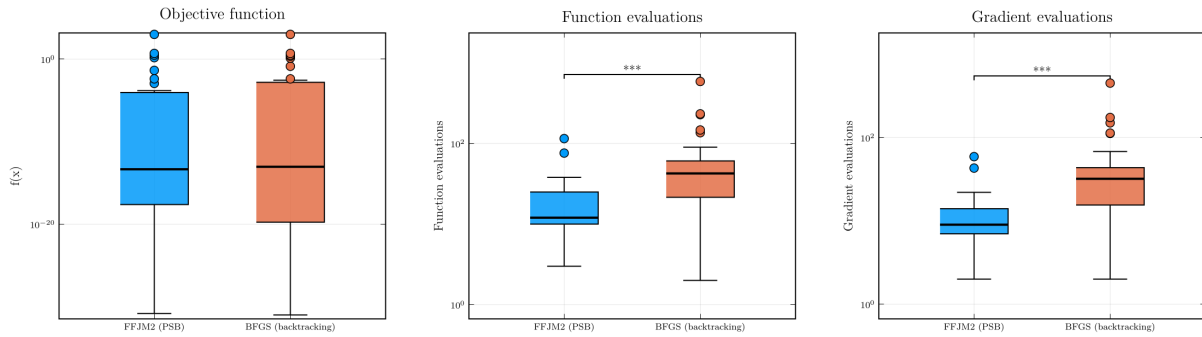


Figure 2: Comparison of Algorithm 4.1 (PSB), called `ffjm2`, vs. BFGS (backtracking) on the MGH problems, restricted to runs where both methods converged by gradient norm. Final f value, function evaluations, and gradient evaluations. Stars indicate significance under the Mann-Whitney U test ($p < 0.05$: *; $p < 0.01$: **; $p < 0.001$: ***).

{fig:boxplot-ffjm2}

As Figure 2 shows, the objective function values are not very different between the two methods. The p-value confirms neither achieves a significantly better result. Function evaluations and gradient

evaluations, however, are different, and the p-value confirms this difference is significant. Algorithm 4.1 therefore performs better than BFGS in this respect, reaching a comparable solution at a much lower cost.

The boxplot above is restricted to the subset of problems where both methods converge by gradient norm. To compare the two methods over all 34 problems, Figure 3 shows the performance profile of Dolan and Moré [15] for the final objective value, function evaluations, and gradient evaluations. For each problem, a method’s ratio r is its evaluation count divided by the smaller of the two methods’ counts on that same problem, so the faster method has $r = 1$. A run that does not converge by gradient norm counts as a failure, with $r = \infty$ (a problem on which neither method converges by gradient norm counts as a mutual failure, $r = \infty$ for both, still included in the 34). The curve $\rho_s(\tau)$ is the fraction of the 34 problems on which method s ’s ratio is at most τ , plotted against $\log_2 \tau$: $\rho_s(1)$ is therefore the fraction of problems on which s is the faster (or tied) method, and $\rho_s(\tau)$ as $\tau \rightarrow \infty$ approaches s ’s overall convergence rate, independent of cost.

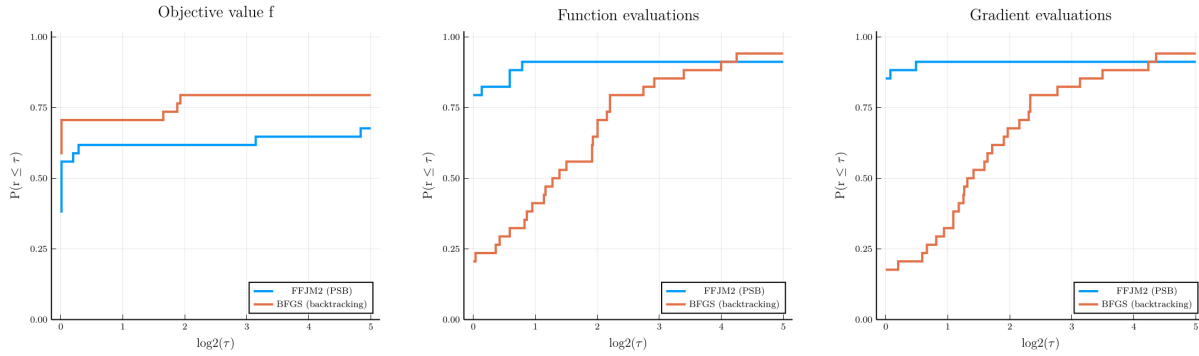


Figure 3: Performance profile [15] of Algorithm 4.1 (PSB), called `ffjm2`, vs. BFGS (backtracking) on all 34 MGH problems, for the final objective value (left), function evaluations (middle), and gradient evaluations (right). $\rho_s(\tau)$ is the fraction of problems on which method s ’s cost (final f value, function evaluations, or gradient evaluations, matching the panel) is at most τ times the smaller of the two methods’ costs on that problem, plotted against $\log_2 \tau$. A run that does not converge by gradient norm counts as a failure.

{fig:performance}

Looking only at the objective value in Figure 3, BFGS comes out ahead, the better (or tied) method on close to 59% of the 34 problems against 38% for Algorithm 4.1. Table 5 and Figure 2, however, already showed that this difference in objective value is not statistically significant. The Mann-Whitney test does not treat it as a real gap in solution quality between the two methods. For function and gradient evaluations, by contrast, Algorithm 4.1 is the better (or tied) method on about 80% of the 34 problems, and that difference *is* significant. Taken together, these results make Algorithm 4.1 an attractive alternative to BFGS. It gives up no solution quality worth mentioning, while reaching that solution at a significantly lower cost.

7 Coefficient estimation problem

{cap:coefficient_}

7.1 The Saint-Venant model and the calibration problem

The Saint-Venant equations [12, 26, 34, 39] describe the flow of fluids in natural channels. The state variables that describe a natural channel vary with respect to $x \in [x_{min}, x_{max}]$ (position, usually measured in meters) and $t \in [t_{min}, t_{max}]$ (time, usually measured in seconds). We define:

- $z(x, t)$: surface elevation
- $h(x, t) = z(x, t) - z_b(x)$: river depth at (x, t)

- $A(x, t)$: cross-sectional wetted area
- $Q(x, t)$: flow rate
- $P(x, t)$: wetted perimeter
- $R(x, t) = A(x, t)/P(x, t)$: hydraulic radius
- g : acceleration of gravity (approximately 9.81m/s^2)

The Saint-Venant equations consist of two main components: Equation (49) describes mass conservation and Equation (50) represents conservation of the linear momentum.

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0 \quad (49) \quad \{\text{mass}\}$$

and

$$\frac{\partial Q}{\partial t} + \frac{\partial}{\partial x} \left(\frac{Q^2}{A} \right) + gA \frac{\partial z}{\partial x} + \frac{n_g^2 Q |Q|}{AR^{4/3}} = 0. \quad (50) \quad \{\text{momentum}\}$$

The Manning (also called Yen-Manning [5]) coefficient n_g represents roughness [5]. Typically, given the inlet discharge $Q(x_{min}, t)$ for $t \in [t_{min}, t_{max}]$, initial conditions $A(x, t_{min})$ and $Q(x, t_{min})$, and the coefficient n_g , numerical methods compute the values of $Q(x, t)$ and $A(x, t)$ at a set of discrete “grid points” (x_ν, t_ν) . This PDE system is solved using an explicit finite difference method with artificial diffusion of 0.99, following [17, 26, 34]. Unfortunately, the Yen-Manning coefficient n_g is often unknown for natural channels and can vary with position x and even time t . For more details, see [5].

Following [19], this research assumes that the Manning coefficient n_g varies with position x , and that a set of elevation observations $z_{obs}(x_\nu, t_\nu)$ are available at n_{obs} distinct locations and times, indexed by $\nu = 1, \dots, n_{obs}$. Consequently, the spatial distribution of $n_g(x)$, discretized over a suitable grid, becomes an unknown component of our problem. Furthermore, to ensure physically plausible results, minimal variation will be required for a function $n_g(x)$ that minimizes spatial oscillations while fitting the solution of the Saint-Venant equations to the observations.

The elevation observations used in this section come from a 3.3 km reach of the East Fork River, Wyoming, USA, described in [29]. Elevation data $z(x, t)$ were recorded at two stations, $x = 751$ m and $x = 3256$ m, every 4 hours from May 20 to June 17, 1977 (three days after the initial conditions), yielding 168 observations per station and a total of $n_{obs} = 336$ elevation observations. Figure 4 illustrates the reach and the location of the two stations.

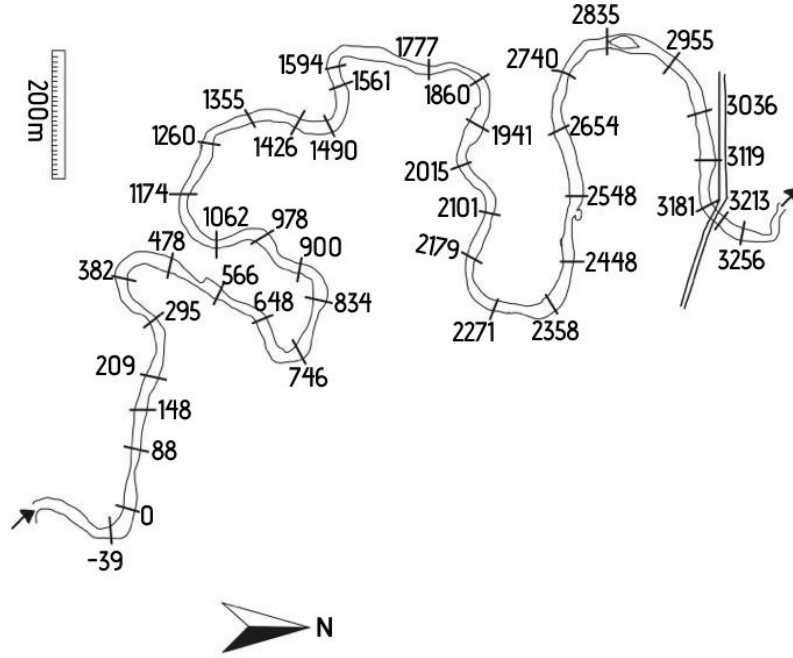


Figure 4: Illustration of the 3.3 km stretch of the East Fork River considered in the model, with markings of the data collection stations used for boundary and initial conditions, recorded between May 17 and June 17, 1977, as available in [29]. Image modified from [4].

{fig: east-fork-m

Consider a grid with n_{grid} points $x_{min} = x_1 < \dots < x_{n_{grid}} = x_{max}$. Collect the n_{grid} values $n_g(x_1), \dots, n_g(x_{n_{grid}})$ into a parameter vector $\xi \in \mathbb{R}^{n_{grid}}$, with $\xi_i := n_g(x_i)$ for $i = 1, \dots, n_{grid}$, kept notationally distinct from the spatial coordinate x . In [19], this calibration problem is posed as a constrained optimization problem, minimizing either the sum of squared deviations of ξ from a reference value $\bar{n}$ or the sum of absolute deviations, subject to equality constraints requiring the simulated elevations to match the n_{obs} observations and a box constraint $0 \leq \xi_i \leq n_g^{max}$. It is solved there by a Safeguarded Augmented Lagrangian method (Algorithm 2.1 of [19]), with BOBYQA or PRIMA handling the subproblems.

The two applications differ mainly in how the underlying PDE is solved. In this work we rely on a more stable PDE solver, which lets us compute the derivative of the objective with respect to ξ accurately. This is precisely what enables the gradient-based comparison carried out below, namely BFGS and Algorithm 4.1 against the derivative-free BOBYQA. Figure 5 contrasts the previous and the current solver on the same problem.

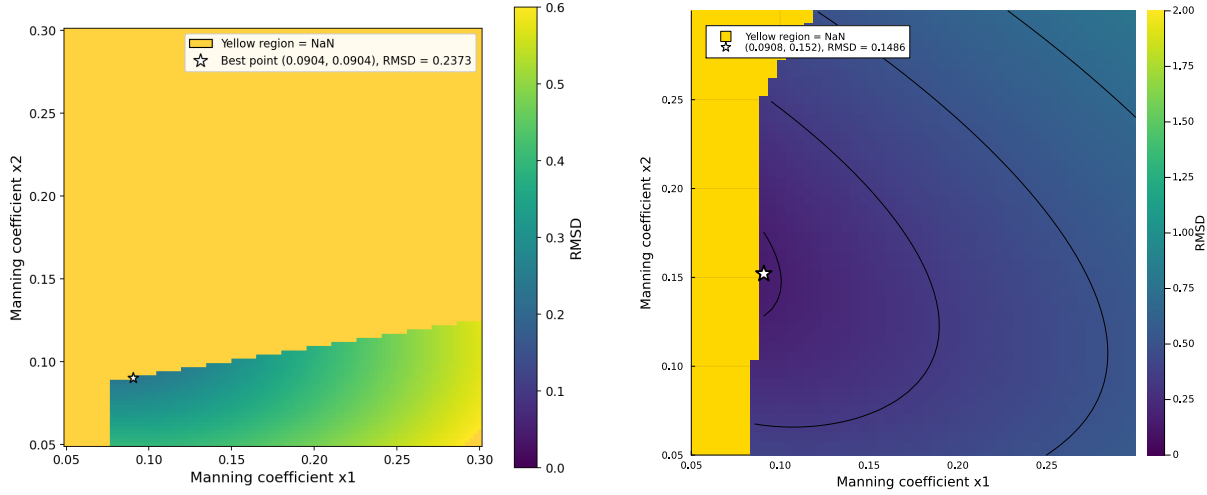


Figure 5: Comparison between the previous (left) and the current (right) PDE solver.

{fig:solver-comp

For the comparison in this section, we adapt this into the unconstrained nonlinear least-squares form used throughout this report, minimizing the elevation mismatch directly instead of enforcing it as an equality constraint. For $i = 1, \dots, m$ (with $m = n_{obs}$), let $z_i(\xi)$ denote the surface elevation the Saint-Venant solution predicts, for parameter vector ξ , at the i -th observation's location and time (x_i, t_i) , and let $z_{obs,i} = z_{obs}(x_i, t_i)$ be the corresponding observation. Define the residual components $f_i(\xi) = z_i(\xi) - z_{obs,i}$, collect them into $F(\xi) = (f_1(\xi), \dots, f_m(\xi))^T$, and minimize

$$f(\xi) = \frac{1}{2} \|F(\xi)\|_2^2 = \frac{1}{2} \sum_{i=1}^m f_i(\xi)^2.$$

Calibrating n_g against the observations is thus the nonlinear least-squares problem of finding ξ that minimizes $f(\xi)$ (ideally $F(\xi) = 0$, though the real data need not be exactly consistent with the model).

7.2 Experimental setup

To calibrate ξ against this residual, four benchmark scenarios compare the same three solvers used throughout this report, namely BFGS (backtracking, order 2), BOBYQA, and Algorithm 4.1, applied directly to $F(\xi)$ above where each solver stands in as the whole optimizer, rather than as Algorithm 4.1's internal subproblem solver. Two scenarios use pregenerated data, where the observations $z_{obs,i}$ are themselves generated by the model at a known ξ^* , so the true minimizer is known exactly. The other two use the real observed data, where the true minimizer (if it exists) is unknown. This real data is the one described previously, obtained from [29]. BFGS and BOBYQA are both given the same box, $[0, 0.5]^{n_{grid}}$, and for the same reason, since a trial step that is too large is likely to land in a forbidden region P (in the terminology of [19], this algorithm's own reference), a region where the PDE simulation diverges and the residual is undefined, so evaluating F there is meaningless and wasted. BFGS enforces the box softly, through an additive penalty term, $\frac{1}{2} \|F(\xi)\|_2^2 + \text{penalty}(\xi)$, where penalty (weight $+\infty$) evaluates to exactly 0 whenever every coordinate of ξ is feasible, as is the case at every minimizer reported below. F is still evaluated everywhere, but the penalty discourages the line search from accepting a step that reaches too far. BOBYQA instead enforces the same box as a hard constraint, through NLOpt's own bound support, so it minimizes $\frac{1}{2} \|F(\xi)\|_2^2$ unpenalized and NLOpt never evaluates the objective at a trial point outside the box in the first place. Algorithm 4.1 minimizes $f(\xi) = \frac{1}{2} \|F(\xi)\|_2^2$ directly, with no box at all, relying instead on its own σ -regularization to keep steps away from that same region.

7.2.1 Algorithmic settings

All three solvers are implemented in Julia, version 1.10, and every experiment reported in this section was run on a computer with a 5.2 GHz 12th Gen Intel(R) Core(TM) i9-12900K processor and 16 GB of 3200 MHz DDR4 RAM, running Linux Mint 21.1. They also share the same evaluation budget, since each residual evaluation is a full transient simulation of the Saint-Venant equations (BFGS and BOBYQA allowed at most 1000 evaluations of the residual, Algorithm 4.1 at most 1000 outer iterations), and BFGS and Algorithm 4.1 further share a gradient-norm convergence tolerance of 10^{-3} ; the remaining settings differ per solver, as detailed below. The Status column in Tables 6, 7, 8, and 9 reports each solver’s own termination code directly, `grad_conv` (Algorithm 4.1 or BFGS) and `Trust_Region` (BOBYQA) marking a stopping criterion that actually certifies stationarity, `reg_stalled` (Algorithm 4.1’s σ stuck at its ceiling) and `step_conv` (BFGS’s line search stuck at its minimum step) marking a stall instead, and `maximum_iterations`, `XTOL_REACHED`, and `FTOL_REACHED` marking the budget above running out without either.

- **BFGS**, the BFGS method from the `Optim.jl` package (version 2.0), together with the order-2 backtracking line search from the `LineSearches.jl` package (version 7.7), the same combination used in Section 6.1. Gradients are obtained by forward-mode automatic differentiation with the `ForwardDiff.jl` package (version 1.3), as discussed above. The line search gives up, reporting `step_conv`, once its step length shrinks below 10^{-12} , and the number of gradient evaluations is capped at 500.
- **BOBYQA**, the implementation provided by the `NLopt.jl` package (version 1.2). The trust region starts with radius 0.01 and the search stops once it has shrunk to 10^{-6} . The number of interpolation points and other internal settings are left at the package’s own defaults.
- **Algorithm 4.1**, our own Julia implementation, like the Saint-Venant solver itself. It uses the same regularization schedule described in Section 6.1, starting at $\sigma_0 = 1$ and growing by a factor of $\sigma_{grow} = 10$ on every rejected step, with a floor of $\sigma_{min} = 10^{-8}$ and a ceiling of $\sigma_{max} = 10^{12}$ kept purely for numerical stability.

7.3 Results

Table 6: Pregenerated-data experiment, $n_{grid} = 2$: $\xi^* = (0.20, 0.15)$, $\xi^0 = (0.25, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	7.47×10^{-7}	9.44×10^{-11}	1.48×10^{-3}	16	9	384.2	<code>grad_conv</code>
BOBYQA	4.61×10^{-8}	3.59×10^{-13}	6.94×10^{-5}	30	0	326.3	<code>XTOL_REACHED</code>
Algorithm 4.1	5.86×10^{-11}	5.80×10^{-19}	6.65×10^{-8}	10	5	158.0	<code>grad_conv</code>

{tab:pregenerated}

All three methods converge to essentially the exact reference $\xi^* = (0.20, 0.15)$. Algorithm 4.1 is the cheapest of the three by every cost measure (fewest evaluations, least time) despite also reaching the tightest gradient norm and RMSD.

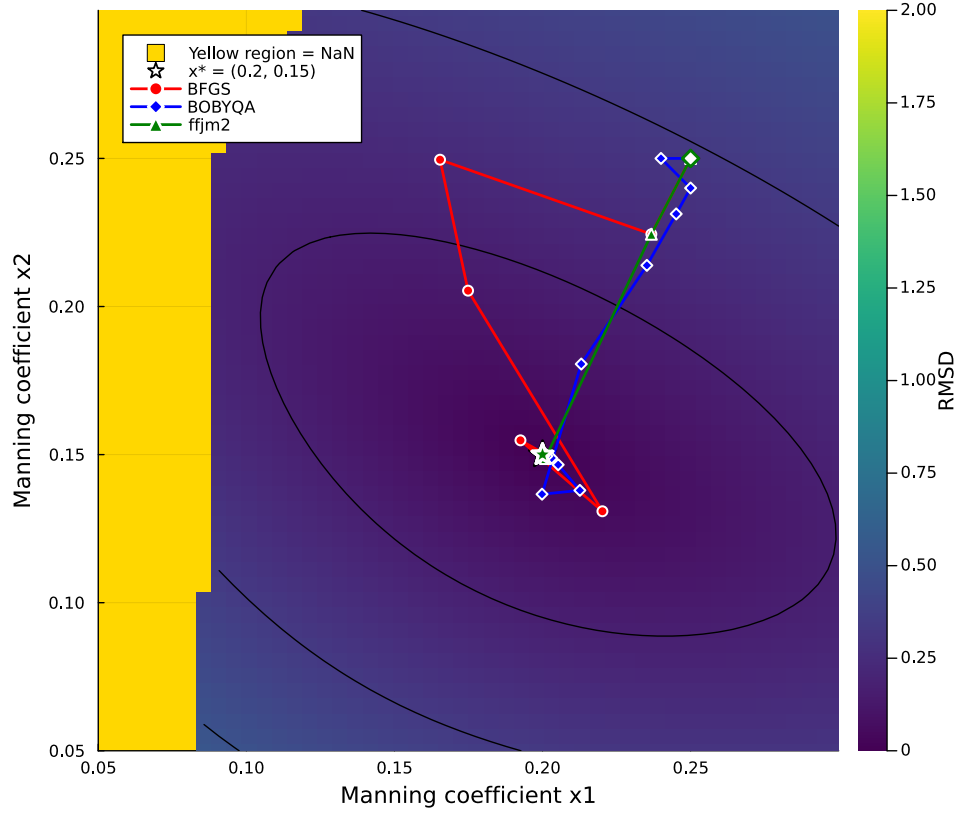


Figure 6: RMSD heatmap of the pregenerated-data experiment ($n_{grid} = 2$) with the path of accepted iterates of each method overlaid: BFGS (red), BOBYQA (blue), and Algorithm 4.1 (green), called **ffjm2**. The white diamond marks the shared starting point $\xi^0 = (0.25, 0.25)$. The star marks each method's final iterate.

{fig:performance_1}

Table 7: Pregenerated-data experiment, $n_{grid} = 10$: decreasing profile ξ^* from 0.20 to 0.15, $\xi^0 = (0.25, \dots, 0.25)$.

{tab:pregenerated}

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	2.00×10^{-1}	6.73×10^0	3.33×10^2	166	28	2453.6	step_conv
BOBYQA	3.68×10^{-4}	2.29×10^{-5}	1.26×10^{-1}	1000	0	12484.1	MAXEVAL_REACHED
Algorithm 4.1	4.38×10^{-6}	3.24×10^{-9}	1.02×10^{-3}	15	9	444.3	grad_conv

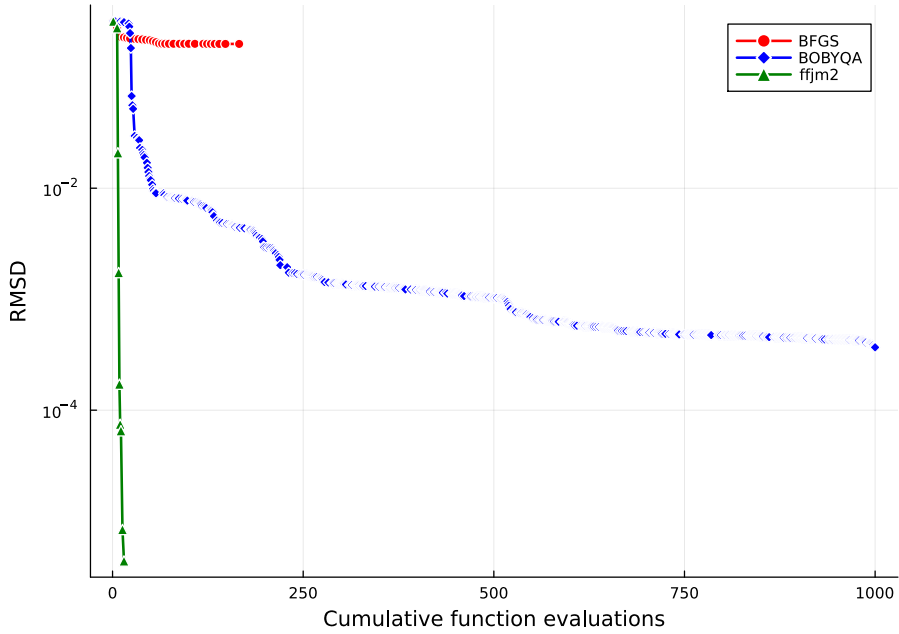


Figure 7: RMSD (log scale) against the cumulative number of function evaluations, for each method on the pregenerated-data experiment ($n_{grid} = 10$): BFGS (red), BOBYQA (blue), and Algorithm 4.1 (green), called `ffjm2`. Each marker is one accepted iterate, in the order it was accepted.

{fig:rmsd-vs-eval}

At $n_{grid} = 10$ Algorithm 4.1 is now the clear winner. It is the only method that certifies a genuine stopping criterion (`grad_conv`, $\|\nabla f\| \approx 1.02 \times 10^{-3}$), and it does so at a small fraction of the cost of the other two (15 function and 9 gradient evaluations, 444.3s). BFGS's line search stalls (`step_conv`) with the worst RMSD and gradient norm of the three (2.00×10^{-1} and $\approx 3.33 \times 10^2$, respectively). BOBYQA gets much closer (RMSD $\approx 3.68 \times 10^{-4}$, $\|\nabla f\| \approx 1.26 \times 10^{-1}$) but exhausts its evaluation budget (`MAXEVAL_REACHED`, 1000 function evaluations) before certifying convergence. At $n_{grid} = 2$ all three methods converged, with Algorithm 4.1 already the cheapest of the three.

Table 8: Real data, $n_{grid} = 2$: $\xi^0 = (0.25, 0.25)$.

{tab:real-dim2}

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	2.10×10^{-1}	7.44×10^0	5.31×10^3	228	32	3191.3	<code>step_conv</code>
BOBYQA	1.42×10^{-1}	3.41×10^0	1.65×10^2	67	0	722.7	<code>Trust_Region</code>
Algorithm 4.1	1.42×10^{-1}	3.42×10^0	1.80×10^2	53	21	806.8	<code>reg_stalled</code>

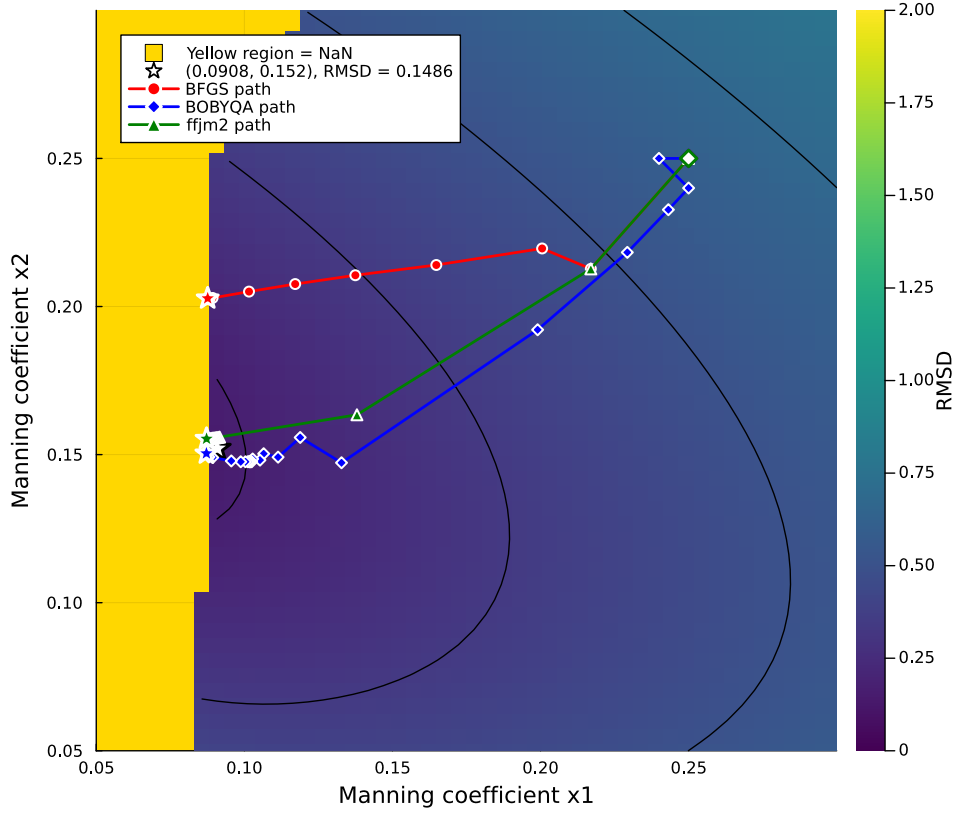


Figure 8: RMSD heatmap of the real-data experiment ($n_{grid} = 2$) with the path of accepted iterates of each method overlaid: BFGS (red), BOBYQA (blue), and Algorithm 4.1 (green), called `ffjm2`. The white diamond marks the shared starting point $\xi^0 = (0.25, 0.25)$, the star marks each method's final iterate.

{fig:performance_1}

On real data, BFGS's line search stalls (`step_conv`) before reaching a good point, with the worst RMSD of the three (2.10×10^{-1}) and a gradient norm still at $\approx 5.31 \times 10^3$. BOBYQA and Algorithm 4.1 land on essentially the same minimizer instead ($\xi \approx (0.087, 0.155)$), with matching RMSD (1.42×10^{-1}) and $f \approx 3.41$ – 3.42 . BOBYQA certifies its own `Trust_Region` criterion using fewer total evaluations and less wall-clock time (722.7s versus 806.8s), while Algorithm 4.1 again stalls on σ (`reg_stalled`) just short of that same point.

Table 9: Real data, $n_{grid} = 10$: $\xi^0 = (0.25, \dots, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	5.15×10^{-2}	4.49×10^{-1}	1.26×10^2	221	61	6253.5	<code>step_conv</code>
BOBYQA	4.68×10^{-2}	3.70×10^{-1}	8.67×10^{-1}	772	0	9120.5	<code>Trust_Region</code>
Algorithm 4.1	4.58×10^{-2}	3.54×10^{-1}	7.08×10^{-1}	60	24	1258.2	<code>reg_stalled</code>

{tab:real-dim10}

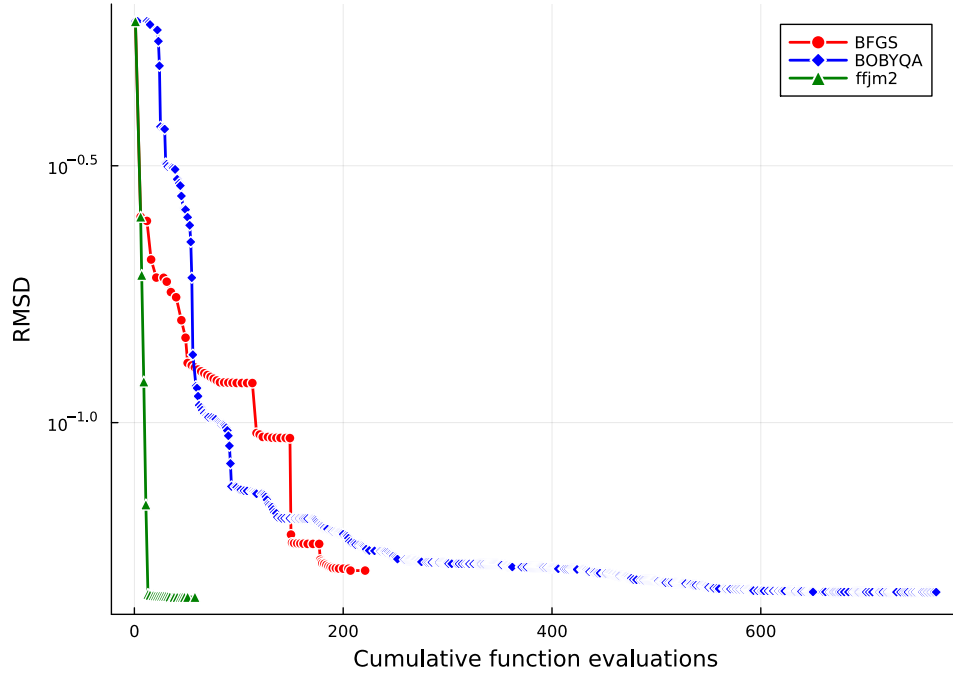


Figure 9: RMSD (log scale) against the cumulative number of function evaluations, for each method on the real-data experiment ($n_{grid} = 10$): BFGS (red), BOBYQA (blue), and Algorithm 4.1 (green), called `ffjm2`. Each marker is one accepted iterate, in the order it was accepted.

{fig:rmsd-vs-eval}

This remains the hardest of the four scenarios: none of the three methods certifies a genuine stopping criterion. BFGS’s line search stalls (`step_conv`) with the largest gradient norm of the three (1.26×10^2) and the worst RMSD (5.15×10^{-2}). BOBYQA certifies its own `Trust_Region` criterion, using 772 evaluations and 2.5h to reach RMSD near to 4.68×10^{-2} . Algorithm 4.1’s regularization parameter again hits its ceiling (`reg_stalled`), but this time it is the best of the three by every measure: the smallest gradient norm (7.08×10^{-1} , even below BOBYQA’s), the lowest RMSD (4.58×10^{-2}), and by far the lowest cost (60 residual and 24 Jacobian evaluations, 21min, against BOBYQA’s 772 evaluations and BFGS’s 282). This is the opposite of the $n_{grid} = 10$ pregenerated-data scenario (Table 7), where the same stall left Algorithm 4.1 the worst of the three, here it is both the cheapest and the most accurate. All three still land on different minimizers with comparable RMSD ($4.6\text{--}5.2 \times 10^{-2}$), and none certifies first-order optimality.

8 Conclusion

{conclusions}

This paper addresses two difficulties that often arise together in optimization problems. Evaluating the objective function, and especially its gradient, may be computationally expensive, so an optimization algorithm must make the most of every such evaluation. At the same time, the domain of the objective function may be restricted by hidden constraints, unknown in advance and not always expressible via equations or inequalities, since the underlying simulation may simply blow up in certain regions, leaving the residual undefined there. Parameter estimation for models governed by partial differential equations is one important class of problems where both difficulties arise, and the focus of this paper, though the difficulties themselves are more general and arise whenever calibrating any expensive-to-solve system of equations.

Sections 2 and 3 handle the first difficulty on its own, for a general smooth unconstrained objective. Algorithm 3.1 adapts a classical line-search scheme by treating a forbidden trial point exactly as a

rejected one, without ever using a function value that does not exist. Every iteration is well defined (Lemma 3.1), and every limit point of the generated sequence is either a forbidden point or a stationary point (Theorem 3.1). Notably, this guarantee requires no Lipschitz or Hölder condition on ∇f . Prior treatments of such hidden constraints are largely derivative-free, such as the extreme-barrier rule used in Mesh Adaptive Direct Search [2, 3, 8, 25, 38], some of them built around a line-search structure [16, 22], and come with little accompanying convergence theory. Algorithm 3.1 extends the same idea, of treating a forbidden trial point as a plain rejected one within a backtracking line search, to the gradient-based case, with an explicit convergence guarantee that fills that theoretical gap.

Section 4 turns to the second difficulty, proposing Algorithm 4.1. At each iteration it minimizes a quartic model built from per-residual quasi-Newton Hessians, regularized by a parameter σ . Unlike Algorithm 3.1, it requires no line search at all. A rejected trial step only increases σ and re-solves the resulting subproblem, rather than rescaling a fixed direction along a ray. It is also independent of the value assigned to a forbidden point, since feasibility is checked directly, before that value could ever enter a comparison, so the same guarantees hold whether such a point is represented by $+\infty$ or by an arbitrarily large finite penalty. Under a Lipschitz-type condition on the residual gradients (Assumption 4.1), Algorithm 4.1 is well defined (Lemma 4.1), drives $\|s^k\| \rightarrow 0$ (Theorem 4.2), and reaches $\|\nabla f(x^k)\| \leq \varepsilon$ in at most $O(\varepsilon^{-2})$ iterations whenever x^k stays a fixed distance away from the forbidden set (Theorems 4.3–4.4 and Corollary 4.1).

Section 5 identifies and resolves a degeneracy specific to this scheme. At the very first iteration ($k = 0$) no quasi-Newton curvature has yet been observed, so the quartic model collapses to a damped Gauss–Newton quadratic. We show that its minimizer is a descent direction for every $\sigma > 0$ (Lemma 5.1.1) and becomes asymptotically parallel to $-\nabla f(x^0)$ as $\sigma \rightarrow \infty$ (Proposition 5.1.2), so the regularization ladder already behaves like a backtracking line search along $-\nabla f(x^0)$ once σ is large enough, with no separate safeguard required. This matters in practice because σ is never reset between outer iterations, only carried over up to a constant factor. Since no curvature information is yet available at $k = 0$, an unnecessarily large σ forced at that very first iteration would persist into later iterations and dampen them more than necessary.

Section 6 calibrates Algorithm 4.1 on the 34 MGH sum-of-squares problems [31], settling on the PSB Hessian update and an exact Newton step for the quartic subproblem, and on an order-2 backtracking line search for the BFGS baseline it is compared against. Over all 34 problems, the two reach statistically indistinguishable final objective values (Mann-Whitney test), while Algorithm 4.1 is significantly cheaper, the better or tied method on about 80% of the problems.

Section 7 takes Algorithm 4.1, BFGS, and the derivative-free BOBYQA to a real engineering problem, calibrating the spatially varying Manning coefficient of the Saint-Venant equations from elevation observations, across four scenarios (synthetic and real data, at dimension 2 and 10). Algorithm 4.1 is the cheapest and most accurate of the three in three of the four scenarios. The exception is the synthetic dimension-10 case, where its regularization parameter σ hits its ceiling and it becomes the worst of the three instead. The same stall reappears on real data at dimension 10, the hardest of the four scenarios, but there it costs nothing: Algorithm 4.1 is again the cheapest and most accurate.

Taken together, these results show that Algorithm 4.1 is efficient and competitive at solving both classical test problems and real engineering problems, offering a practical alternative to BFGS and BOBYQA for expensive sum-of-squares problems defined on a domain with forbidden regions, and typically the cheapest of the three whenever it converges.

A weakness of the method is that its surrogate is a quartic model, so global optimality of the subproblem itself cannot be guaranteed, only an approximate global solution obtained by multistart. However, note that global solution of the subproblem is not needed at all in the definition of the algorithm. Moreover, once this model no longer represents the objective function well, it must be reset entirely rather than merely updated, which can be a problem when the minimization starts far from a solution of a strongly non-quadratic objective.

For future work, we intend to better characterize when the model stops representing the objective function well. Currently this is judged in aggregate, through the ratio ρ_k between actual and predicted reduction in f , which triggers a reset of every H_i at once. Examining each residual f_i individually seems

like a promising alternative, and could guide decisions to further improve the proposed algorithm. We also intend to apply the algorithm to larger and more complex problems, where its efficiency advantage may become even more evident.

References

- [1] A. Agresta, M. Baiocchi, C. Biscarini, and A. Milani, Evolutionary Algorithms for Roughness Coefficient Estimation in River Flow Analyses, in *Applications of Evolutionary Computation (EvoApplications 2021)*, Lecture Notes in Computer Science 12694, Springer, pp. 795–811, 2021.
- [2] Ch. Audet and J. E. Dennis Jr. Mesh adaptive direct search algorithms for constrained optimization, *SIAM Journal on Optimization* 17, pp. 188–217, 2006.
- [3] Ch. Audet, J. E. Dennis Jr., and S. Le Digabel, Globalization strategies for Mesh Adaptive Direct Search, *Computational Optimization and Applications* 46, pp. 193–215, 2010.
- [4] M. T. Ayvaz, A linked simulation–optimization model for simultaneously estimating the Manning’s surface roughness values and their parameter structures in shallow water flows, *Journal of Hydrology* 500, pp. 183–199, 2013.
- [5] E. G. Birgin, M. R. Correa, V. A. González-López, J. M. Martínez, and D. S. Rodrigues, Randomly supported variation of deterministic models and its application to one-dimensional shallow water flows, *Journal of Hydraulic Engineering* 150(5): 04024026, 2024.
- [6] E. G. Birgin and J. M. Martínez, Accelerated derivative-free nonlinear least-squares applied to the estimation of Manning coefficients, *Computational Optimization and Applications* 81, pp. 689–715, 2022.
- [7] E. G. Birgin and J. M. Martínez, *Practical Augmented Lagrangian Methods for Constrained Optimization*, Society for Industrial and Applied Mathematics, Philadelphia, 2014.
- [8] A. Brilli, A. L. Custódio, G. Liuzzi, and E. J. Silva, Nonlinear derivative-free constrained optimization with a penalty-interior point method and direct search, *Numerical Algorithms*, 2026, <https://doi.org/10.1007/s11075-026-02360-5>.
- [9] C. G. Broyden, The convergence of a class of double-rank minimization algorithms 2. The new algorithm, *IMA Journal of Applied Mathematics* 6, pp. 222–231, 1970.
- [10] C. Cartis, N. I. M. Gould, and Ph. L. Toint, Adaptive cubic regularisation methods for unconstrained optimization. Part I: motivation, convergence and numerical results, *Mathematical Programming* 127(2), pp. 245–295, 2011.
- [11] C. Cartis, N. I. M. Gould, and Ph. L. Toint, Adaptive cubic regularisation methods for unconstrained optimization. Part II: worst-case function- and derivative-evaluation complexity, *Mathematical Programming* 130(2), pp. 295–319, 2011.
- [12] O. Castro-Orgaz and W. H. Hager, *Shallow Water Hydraulics*, Springer, 2019.
- [13] A. R. Conn, K. Scheinberg, and L. N. Vicente. *Introduction to Derivative-Free Optimization*, MPS-SIAM Series on Optimization, SIAM, Philadelphia, 2009.
- [14] J. E. Dennis, Jr. and H. Wolkowicz, *Sizing and least-change secant methods*, SIAM J. Numer. Anal., 30 (1993), pp. 1291–1314.
- [15] E. D. Dolan and J. J. Moré, *Benchmarking optimization software with performance profiles*, Math. Program., 91 (2002), pp. 201–213.

- [16] G. Fasano, G. Liuzzi, S. Lucidi, and F. Rinaldi, A line search derivative-free approach for nonsmooth constrained optimization, *SIAM Journal on Optimization* 24, pp. 959–992, 2014.
- [17] D. M. Ferreira, C. V. S. Fernandes, and J. Gomes, Verification of Saint-Venant equations solution based on the Lax diffusive method for flow routing in natural channels, *Revista Brasileira de Recursos Hídricos*, vol. 22, 2017. <https://doi.org/10.1590/2318-0331.011716104>
- [18] R. Fletcher, A new approach to variable metric algorithms, *The Computer Journal* 13, pp. 317–322, 1970.
- [19] F. A. Fortunato and J. M. Martínez, *KKT-free constrained optimization and application to river-flow modelling*, *Optimization Methods and Software*, pp. 1–32, 2026, <https://doi.org/10.1080/10556788.2026.2682934>.
- [20] D. Ge, J. Liu, T. Liu, and J. Tan, 1053: SOLNP+: A Derivative-Free Solver for Constrained Nonlinear Optimization, *ACM Transactions on Mathematical Software* 50(4), DOI:10.1145/3699956, 2024.
- [21] D. Goldfarb, A family of variable-metric methods derived by variational means, *Mathematics of Computation* 24, pp. 23–26, 1970.
- [22] S. Gratton and L. N. Vicente, A merit function approach for direct search, *SIAM Journal on Optimization* 24(4), pp. 1980–1998, 2014.
- [23] H. Hosseiny, Implementation of heuristic search algorithms in the calibration of a river hydraulic model, *Environmental Modelling & Software* 157, 105537, 2022.
- [24] H. Kang, Y. Kim, H. An, J. Byun, and J. Noh, Estimation of river discharge using Monte Carlo simulations and a 1D hydraulic model based on the artificial multi-segmented rating curves at the confluence of two rivers, *Environmental Research Communications* 6(2), 025012, 2024.
- [25] S. Le Digabel and S. M. Wild, A taxonomy of constraints in black-box simulation-based optimization, *Optimization and Engineering* 25, pp. 1125–1143, 2024.
- [26] R. J. LeVeque, *Numerical Methods for Conservation Laws*, Lectures in Mathematics, ETH Zürich, Birkhäuser, 1992.
- [27] H. B. Mann and D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, *Annals of Mathematical Statistics* 18(1), pp. 50–60, 1947.
- [28] J. M. Martínez, Levenberg-Marquardt revisited and parameter tuning of river regression models, *Computational and Applied Mathematics* 43, 14, 2024, <https://doi.org/10.1007/s40314-023-02535-z>.
- [29] R. H. Meade, R. M. Myrick, and W. W. Emmett, Field data describing the movement and storage of sediment in the East Fork River, Wyoming, Part II, *USGS Open-File*, 1979.
- [30] P. K. Mogensen and A. N. Riseth, *Optim: A mathematical optimization package for Julia*, J. Open Source Software, 3 (2018), p. 615.
- [31] J. J. Moré, B. S. Garbow, and K. E. Hillstom, *Testing unconstrained optimization software*, ACM Trans. Math. Software, 7 (1981), pp. 17–41.
- [32] V. T. Nguyen, D. Georges, and G. Besançon, State and parameter estimation in 1-D hyperbolic PDEs based on an adjoint method, *Automatica* 67, pp. 185–191, 2016.
- [33] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd edition, Springer Series in Operations Research and Financial Engineering, Springer, New York, 2006.

- [34] R. M. Porto, *Hidráulica Básica*, EESC-USP, São Paulo, 2000.
- [35] M. J. D. Powell, A new algorithm for unconstrained optimization, in *Nonlinear Programming*, J. B. Rosen, O. L. Mangasarian, and K. Ritter (eds.), Academic Press, New York, NY, pp. 31–65, 1970.
- [36] M. J. D. Powell, Some global convergence properties of a variable metric algorithm for minimization without exact line searches, in *Nonlinear Programming*, SIAM-AMS Proceedings, Vol. 9, R. W. Cottle and C. E. Lemke (eds.), American Mathematical Society, Providence, RI, pp. 53–72, 1976.
- [37] M. J. D. Powell, *The BOBYQA algorithm for bound constrained optimization without derivatives*, Cambridge NA Report NA2009/06, University of Cambridge, Cambridge, 2009.
- [38] T. M. Ragonneau and Z. Zhang, PDFO: a cross-platform package for Powell’s derivative-free optimization solvers, *Mathematical Programming Computation* 16, pp.1-25, 2024.
- [39] A. J. C. Saint-Venant, Théorie du mouvement non-permanent des eaux, avec application aux crues des rivières et à l’introduction des marées dans leur lit, *Comptes Rendus des Séances de Académie des Sciences* 73, pp. 147–154, 1871.
- [40] D. F. Shanno, Conditioning of quasi-Newton methods for function minimization, *Mathematics of Computation* 24, pp. 647–656, 1970.
- [41] K. Ueda and N. Yamashita, On a global complexity bound of the Levenberg-Marquardt method, *Journal of Optimization Theory and Applications* 147, pp. 443–453, 2010.